Nombre	Matrícula
1. Roberto Santana Trujillo 2. Isaías Leonardo Salazar Torres	2227084 2227016
Curso: Componentes de lógica de negocio	Profesor: Miguel Ángel Gómez Marroquín
Módulo: Proyecto integrador	Actividad: PIA — FerreMax
Fecha de entrega: 06/11/2026	Equipo 17

Objetivo

Integrar el análisis de FerreMax de U2/U3 con una aplicación de lógica de negocio verificable. El núcleo permite consultar inventario, conservar versiones de cotización, crear pedidos pendientes, confirmarlos con reserva y cancelarlos liberando únicamente su reserva. Se aplican DDD, Specification, máquina de estados, transacciones, pruebas, Docker Compose y CI/CD.

Este informe reúne BRD, arquitectura, evidencias, conclusiones y anexos de análisis. Distingue procesos implementados, simulados y diseñados. La cobertura de líneas no demuestra por sí sola que todas las historias de U2 estén implementadas.

Repositorio: https://github.com/Santana-R/ferremax-u3

Sistema y Swagger: https://ferremax-api-w5kd.onrender.com/docs

Base funcional documentada: c4876ef. Evidencia registrada: 249 pruebas aprobadas, cero omitidas y cobertura 97.38%. El registro docs/pia/CIERRE.md identifica por separado la integración y el despliegue final.

Procedimiento y recopilación de entregables

Entregable	Contenido y ubicación
BRD y análisis	BRD completo en anexo A; 20 historias con 46 criterios en anexo B; BPMN y glosario en anexos C/D.
Arquitectura	Modelo DDD, cinco vistas UML/arquitectura, patrones, SOLID y siete ADR.
Implementación	Seis operaciones REST, PostgreSQL y reglas transaccionales del núcleo.
Calidad y despliegue	Resultados de CI identificados por commit, Swagger, Docker y consultas públicas.
Conclusiones	Resultados alcanzados, límites y mejoras pendientes.
Repositorio y apoyo	Código, docs, tests, CI/CD, Docker, README y presentación editable/PDF.

1. Problema, alcance y trazabilidad

FerreMax necesita evitar comprometer mercancía inexistente y convertir condiciones comerciales vencidas. Su análisis contempla ventas, crédito, transferencias, surtido, entrega y devoluciones. La implementación actual cubre el recorrido contado de cotización a pedido y reserva/cancelación, conservando las demás necesidades como pendientes explícitos.

Parte	Estado demostrable
Implementado	Disponibilidad; revalidación versionada; conversión aceptada; consulta de pedido; confirmación con reserva; cancelación con liberación.
Simulado	Identidad/roles HTTP fijos y autorización de pago mediante datos de demostración; sin pasarela ni autenticación real.
Parcial o diseñado	Emisión operativa inicial, crédito integral, reducción, multiorigen, transferencias, parciales, surtido, entrega, devolución, panel y consulta protegida de auditoría.

La trazabilidad conecta BN/regla → historia/criterio → actividad BPMN → componente → prueba. Los archivos docs/analysis/trazabilidad_analisis.csv y docs/architecture/trazabilidad_arquitectura.csv identifican tanto cobertura implementada como ausencia de flujo. R16 permanece Parcial y no se presenta una dispensa del profesor.

2. Reglas que se comprueban

Regla	Éxito	Error	Límite
Disponibilidad	Con 3 disponibles reservar 2 deja 1.	Solicitar 4 se rechaza sin modificar pedido ni inventario.	Reservar exactamente 3 deja 0.
Vigencia e historial	Revalidar crea una versión nueva y conserva la anterior.	La versión vencida no crea pedido ni modifica datos.	En el instante del vencimiento ya no es vigente.
Cancelación y reserva propia	Cancelar antes del surtido libera 2 propias y conserva 7 de otro pedido.	Un pedido ya surtido se rechaza y conserva estado/reservas.	Una segunda cancelación no libera ni registra otro evento.

Estas tres reglas tienen nueve casos E9 reutilizados en el PIA. La suite completa añade atomicidad, validación HTTP, datos inválidos, concurrencia, repeticiones y rollback con PostgreSQL real. E9 conserva sus propias métricas históricas; no se sustituyen por las de la suite posterior.

3. Arquitectura y componentes

Este modelo describe el código de src/ferremax, separado del mapa completo de negocio U2/U3. Los diagramas PIA y la trazabilidad muestran esas mismas responsabilidades.

Entidades y límites de agregados del núcleo

Elemento	Identidad y contenido	Comportamiento y límite
Cotizacion	ID y versión, folio, cliente, vigencia, partidas y condiciones	revalidar devuelve una versión nueva; convertir comprueba vigencia/aceptación y crea Pedido. Cada versión se persiste con PK compuesta; no se sobreescribe la anterior.
Pedido	ID, cliente, referencia a ID/folio/versión de cotización, snapshot de partidas, estado, surtido y referencia de pago	confirmar, cancelar, cantidades por SKU y total derivado. Referencia la cotización por identidad, no la posee como objeto hijo.
InventarioUbicacion	SKU + ubicación, física, tránsito y diccionario de reservas por ID de pedido	Calcula comprometida/disponible, valida integridad, reserva y libera. La reserva es una entrada del diccionario y una fila SQL; no existe una clase Python Reserva que deba dibujarse.

Cotización, pedido e inventario se tratan como límites distintos del núcleo. Confirmar/cancelar necesitan consistencia entre más de uno; el servicio abre una transacción local que los coordina. Esta decisión práctica del prototipo no los convierte en un único agregado ni en una garantía de consistencia distribuida.

La copia de partidas conserva las condiciones de la versión aceptada. Las listas del pedido y la cotización son distintas; sus miembros son Value Objects congelados. Los agregados siguen siendo dataclasses públicas

con listas/diccionarios mutables: la disciplina de la aplicación usa métodos y repositorios, pero no hay encapsulación profunda que impida cualquier mutación Python directa.

Value Objects y datos sin identidad operativa propia

Clase	Datos	Invariante o uso real
SKU	valor	Texto no vacío; identifica producto sin implementar un catálogo maestro completo.
Dinero	importe Decimal, moneda	Importe finito no negativo y moneda presente; las partidas de un documento requieren moneda homogénea.
Vigencia	emisión y vencimiento con zona horaria	Vencimiento posterior; vigente en emisión <= ahora < vencimiento.
Partida	número, SKU, cantidad, precio	Número/cantidad positivos, cantidad finita. Validación del conjunto impide números repetidos o mezcla de monedas.
AutorizacionPago	referencia y autorizada	Portador congelado de la decisión simulada. Pedido.confirmar exige autorización verdadera y referencia no vacía; el objeto no conecta a una pasarela.
ContextoConversion	cotización, ahora y aceptada	Contexto congelado que reciben las Specifications. No realiza IO ni crea pedidos.

No se denomina inmutabilidad profunda a una dataclass frozen=True que contiene un diccionario u otro objeto mutable. Este límite aplica a EventoDominio.datos y a la cotización referenciada por ContextoConversion.

Reglas, servicios y Specifications

CotizacionVigente y AceptacionDelCliente heredan de Specification[ContextoConversion]. AndSpec, OrSpec y NotSpec permiten composición. La regla comercial actual utiliza:

```
COTIZACION_CONVERTIBLE = COTIZACION_VIGENTE & ACEPTACION_DEL_CLIENTE
```

La evaluación es pura. La cotización conserva el motivo del rechazo y crea el pedido solo después de cumplir la regla. OR/NOT están implementados y probados como operadores; no se les atribuyen condiciones comerciales adicionales.

CotizacionService y PedidoService son servicios de **aplicación**: verifican actor, abren UoW, coordinan repositorios, invocan el dominio y registran auditoría/eventos. El reloj y la fábrica de UoW se inyectan. Crear versiones, determinar vigencia y proteger transiciones/reserva son responsabilidades del dominio.

PoliticaCredito representa una política de dominio aislada; usa CuentaCredito para condición activa, bloqueo, límite/saldo y moneda. No existe todavía un flujo de crédito, un repositorio de cuenta o una autorización excepcional persistida. La clase no acredita el agregado operativo completo de crédito del diseño U3.

Estados y eventos

La máquina usa EstadoPedido y guardas de métodos, sin una clase State por estado. Sus transiciones ejecutables son Pendiente → Confirmado → Cancelado. surtido es una guarda de cancelación, no una operación de surtido ni un cuarto estado implementado. Ver tabla completa.

Evento	Construcción y persistencia	Referencia
CotizacionRevalidada	El servicio construye EventoDominio con snapshot de la nueva versión y lo registra junto a auditoría	US04; BR02/BR03/BR16
PedidoCreado	Se guarda con el pedido Pendiente derivado de cotización	US05; BR01/BR16
PedidoConfirmado	Se registra con estado, reserva y referencia tras las validaciones; todo se hace durable al commit	US06/US09; BR05/BR16
PedidoCancelado	Se registra con la liberación propia y estado Cancelado en la misma transacción	US10; BR06/BR16

Los servicios construyen estos eventos; los agregados no mantienen una lista propia de eventos. Outbox conserva los registros, sin publicador externo. Un fallo antes del commit no deja un hecho durable de éxito. La auditoría conserva los valores previos/nuevos y el motivo, con actor de demo en HTTP.

Relación entre clases, almacenamiento y API

Concepto	Clase/contrato	Persistencia / entrada
Versiones	Cotizacion, CotizacionRepository	cotizacion_version: clave id/version y snapshot JSONB
Pedido	Pedido, PedidoRepository, PedidoReader	pedido: FK a versión, estado y snapshot JSONB
Inventario y reservas	InventarioUbicacion, InventarioRepository	inventario y reserva, con claves compuestas y cantidades
Dinero/partidas/vigencia	Value Objects	Se serializan dentro de snapshots; no hay una tabla por cada clase
Auditoría/eventos	UnitOfWork.registrar, EventoDominio	auditoria y outbox dentro de la misma transacción
Actores	Actor en aplicación	Roles ficticios construidos por los endpoints; no hay tabla de usuarios

Los modelos Pydantic de api/schemas.py validan el contrato externo; no son raíces de agregados. No se dibuja Producto, Cliente, Transferencia, Entrega o Devolucion como clase operativa de este código si todavía no existe.

Verificación identificable

Comportamiento	Prueba existente
Vigencia, aceptación y composición sin mutar	tests/test_specifications.py
Copia de versión/origen y ausencia de reserva al convertir	tests/test_api_conversion_postgres.py y tests/test_api_revalidacion_postgres.py
Seis combinaciones de estado/acción y guardas	tests/test_state_machine.py
Reserva, última unidad, segunda partida y rollback	tests/test_api_confirmacion_postgres.py
Liberación propia, repeticiones y fallo de segunda liberación	tests/test_api_cancelacion_postgres.py
Total del pedido separado de posteriores precios de cotización	tests/test_api_consulta_pedido_postgres.py

Las pruebas respaldan el comportamiento del núcleo. Las necesidades diseñadas del BRD y los criterios sin prueba integral de US01-US20 conservan su estado pendiente.

3.1 Diagramas que corresponden al código

A01 — Componentes y dependencias

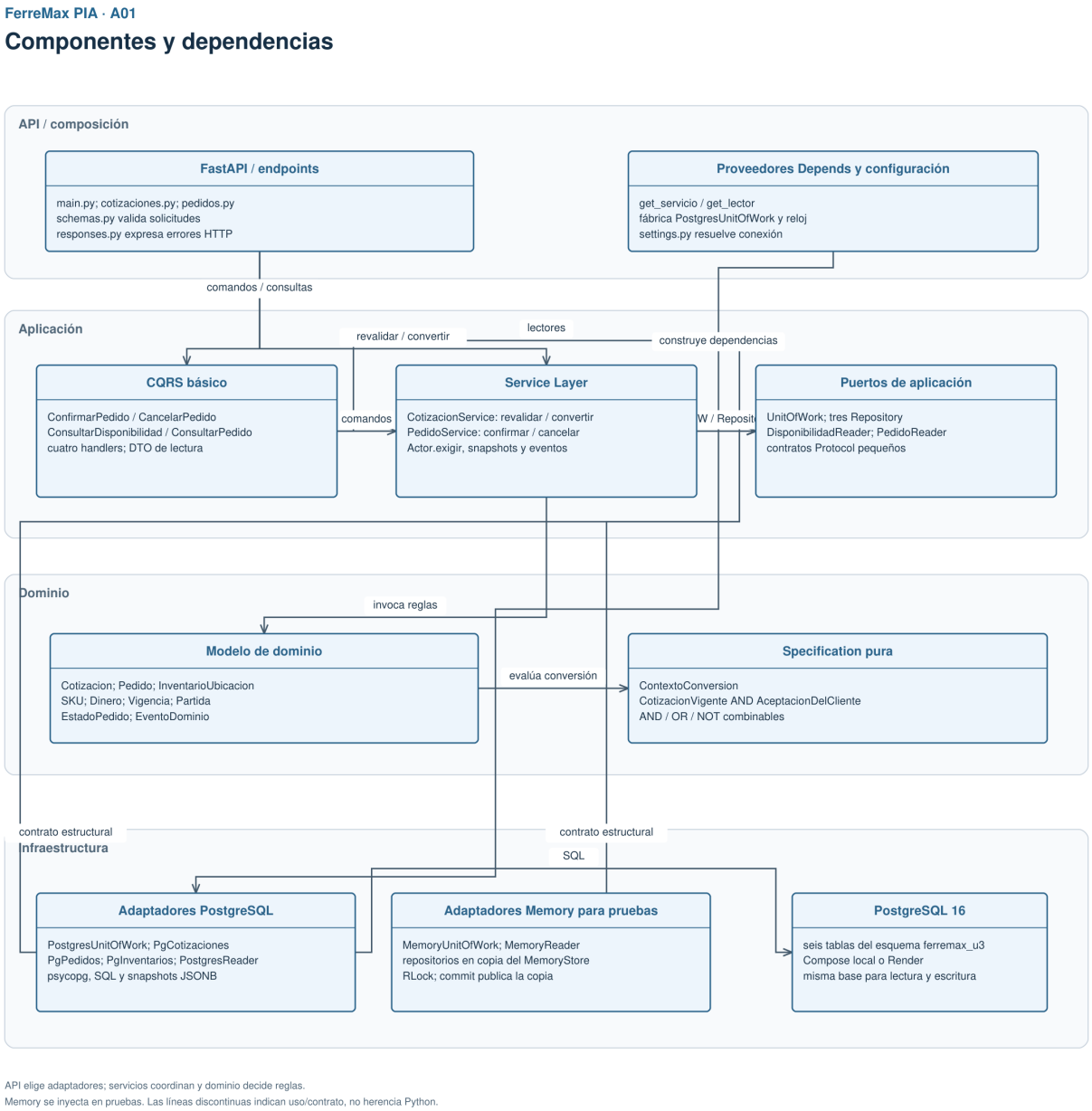


Figura A01. Vista PIA; fuente editable en docs/architecture/diagrams/FerreMax_PIA_Arquitectura.drawio.

A02 — Clases y límites de agregados

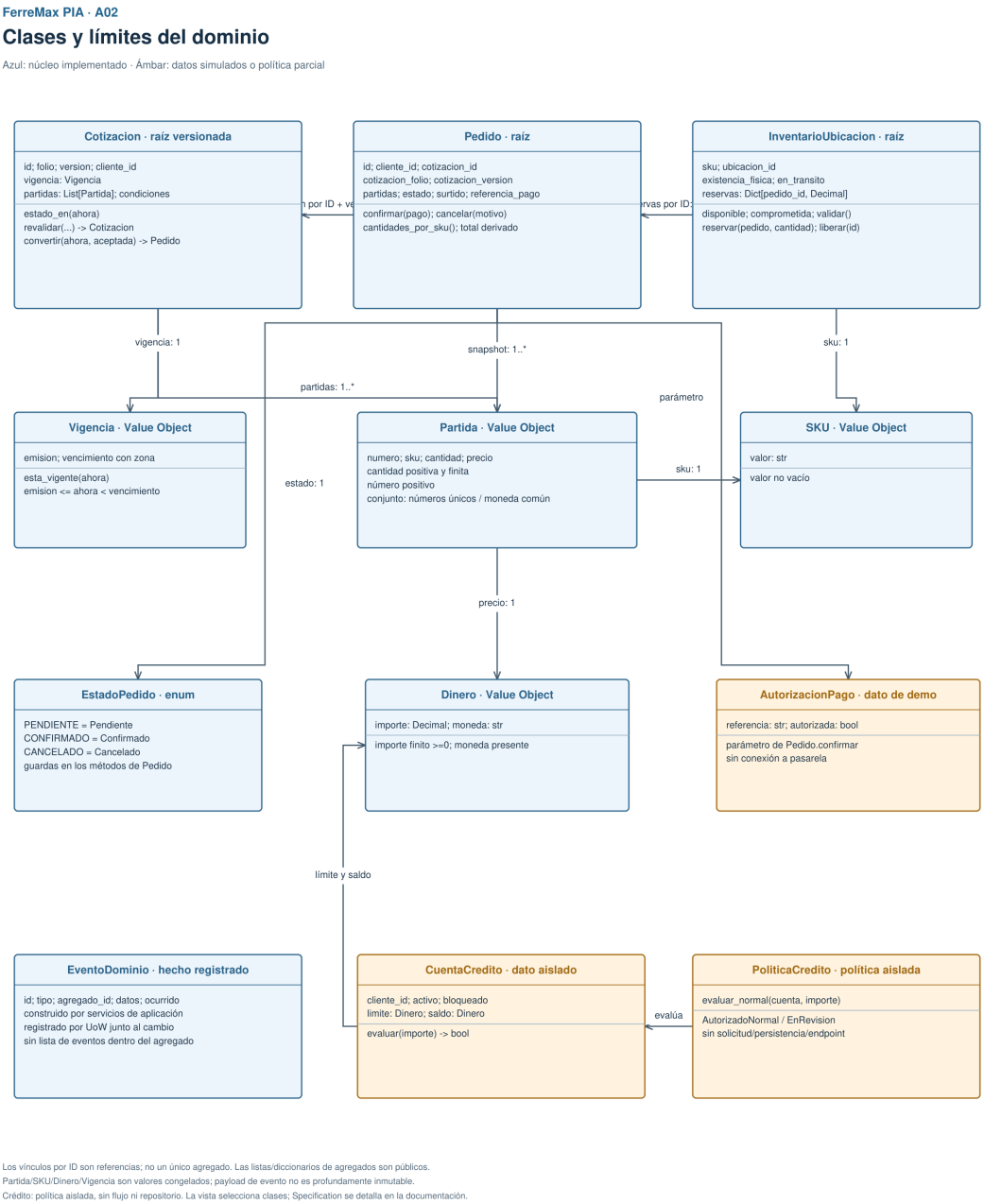


Figura A02. Vista PIA; fuente editable en docs/architecture/diagrams/FerreMax_PIA_Arquitectura.drawio.

A03 — Confirmación válida y commit

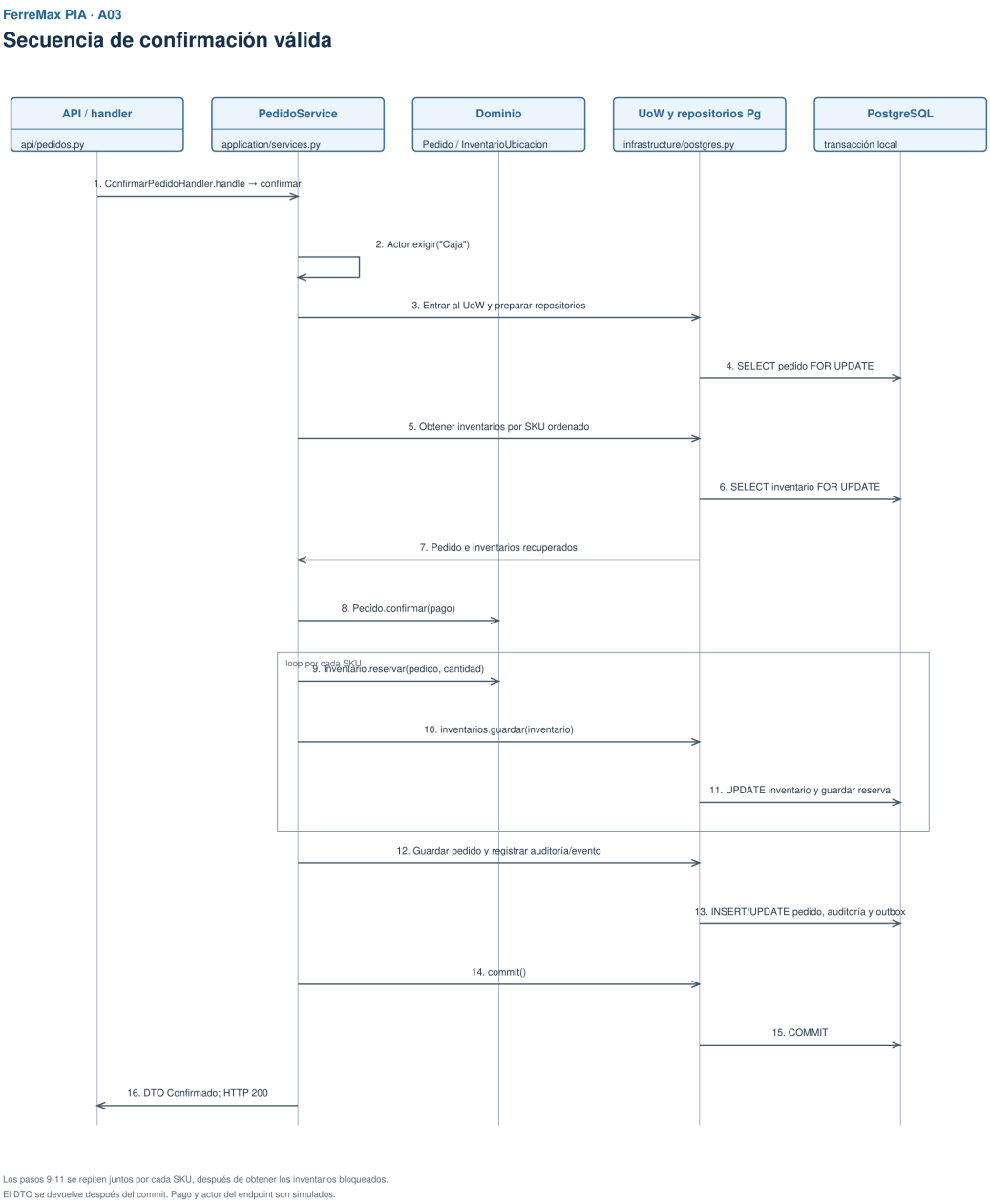


Figura A03. Vista PIA; fuente editable en docs/architecture/diagrams/FerreMax_PIA_Arquitectura.drawio.

A04 — Rechazo de segunda partida y rollback

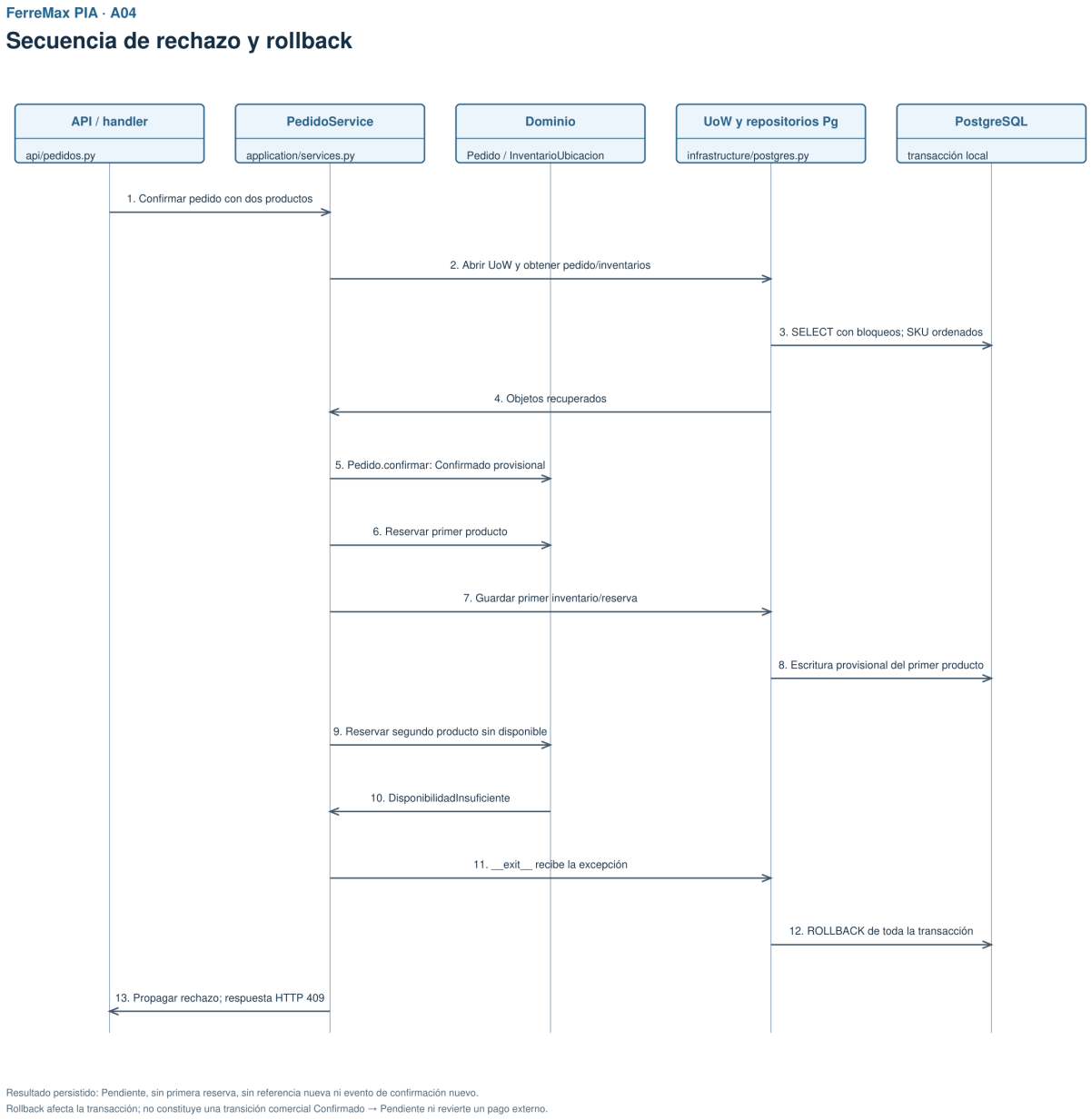


Figura A04. Vista PIA; fuente editable en docs/architecture/diagrams/FerreMax_PIA_Arquitectura.drawio.

El rollback conserva el pedido persistido como Pendiente. No se modela como una transición comercial Confirmado → Pendiente ni se afirma que revierta un pago bancario externo.

A05 — Persistencia y relaciones físicas

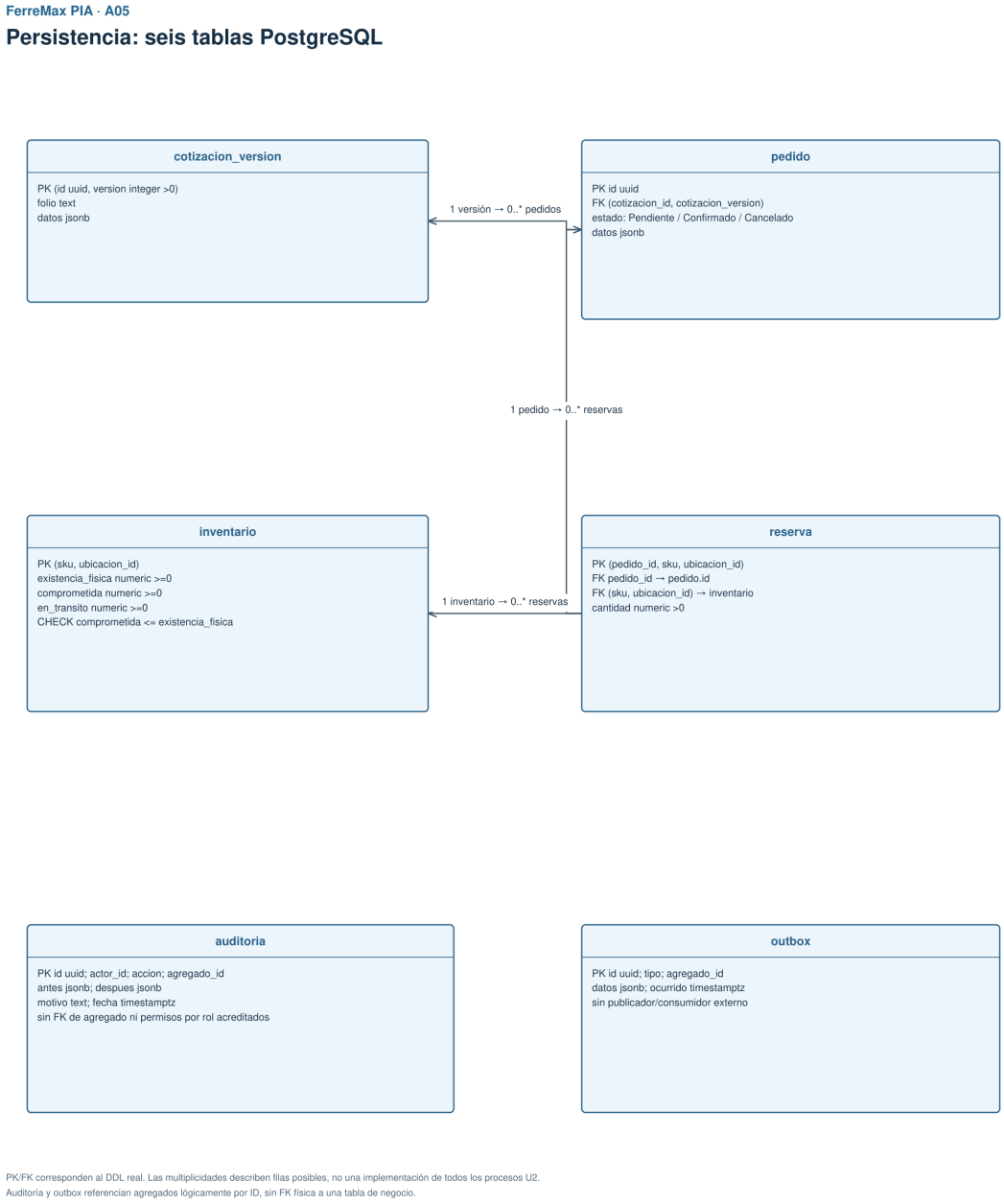


Figura A05. Vista PIA; fuente editable en docs/architecture/diagrams/FerreMax_PIA_Arquitectura.drawio.

Auditoría y outbox registran identificadores lógicos. Solo se dibujan las relaciones físicas existentes. El registro outbox no acredita entrega de mensajes a un consumidor externo.

4. Patrones, SOLID y decisiones

Las decisiones siguientes se justifican por el código actual y los casos de FerreMax. Las pruebas funcionales apoyan el diseño, pero no constituyen una certificación automática de SOLID. Los ADRs registran alternativas y consecuencias.

Patrones utilizados

Patrón	Problema que resuelve	Aplicación concreta	Evidencia y costo
Domain Model	Vigencia, estado y disponibilidad reaparecen en varias operaciones	Cotizacion, Pedido, InventarioUbicacion y valores en domain/model.py	Pruebas de dominio/E9 y API; agrega objetos y exige cuidar encapsulación de colecciones públicas.
Service Layer	Coordinar roles, transacción y colaboradores sin introducir HTTP en el dominio	CotizacionService, PedidoService	Mismos servicios usados con Memory/PostgreSQL; coordina más de un agregado en una transacción local.
Repository	Recuperar/guardar agregados sin SQL dentro del caso de uso	Protocols y adaptadores Memory/Pg de cotizaciones, pedidos e inventarios	Integración PostgreSQL y pruebas de servicios; la interfaz debe conservar semántica de errores y bloqueos.
Unit of Work	Confirmar estado y reserva, o cancelar/liberar, de manera atómica	PostgresUnitOfWork y MemoryUnitOfWork	Segunda partida fallida revierte; añade conexiones, límites de transacción y contención.
CQRS básico	Separar consultas de comandos con responsabilidades y DTO distintos	Cuatro mensajes/handlers en cqrs.py; DisponibilidadReader y PedidoReader	Consultas no escriben; misma base, sin Event Sourcing ni réplica de lectura. Revalidar/convertir usan servicio directamente.
Specification	Nombrar y componer condiciones de elegibilidad puras	Vigencia AND aceptación antes de convertir	test_specifications.py prueba AND/OR/NOT, fronteras y ausencia de cambios; solo AND compone la regla comercial actual.
Máquina de estados	Restringir cambios de ciclo de vida	Enum y guardas en Pedido.confirmar/cancelar	Tabla de tres estados × dos acciones. Es una máquina explícita; no se afirma una jerarquía de clases del patrón GoF State.
Registro outbox transaccional	Conservar el hecho de negocio junto a su cambio	UnitOfWork.registrar escribe auditoría y EventoDominio en outbox	No hay publicador/consumidor ni garantía de entrega externa; ese trabajo sigue pendiente.

Principios SOLID con ejemplos reales

Principio	Cómo se demuestra aquí	Referencia concreta	Límite reconocido
S - responsabilidad única	El dominio determina validez; el servicio coordina; el adaptador persiste; la API traduce a HTTP	Pedido.confirmar, PedidoService.confirmar, PgInventarios.guardar, api/pedidos.py	Los archivos agrupan varias clases; el principio se evalúa por responsabilidades, no por número de archivos. El servicio también arma snapshots/eventos y debe evitar absorber más reglas.
O - abierto a extensión	Las Specifications compuestas y los adaptadores permiten extender condiciones/infraestructura mediante colaboradores	Specification.__and__/__or__/__invert__; Memory/Pg	Agregar una condición todavía requiere decidir cómo se compone y qué mensaje produce; no se promete cero cambios en todo el sistema.
L - sustitución	Servicios y handlers usan operaciones esperadas de UoW/repositorios/lectores, con implementaciones Memory y PostgreSQL	Pruebas de API con Memory y archivos test_api_*_postgres.py con DB real	Las pruebas ofrecen evidencia de escenarios comunes, no una demostración formal universal. RLock global y bloqueos de filas no son mecanismos equivalentes ni rinden igual.
I - segregación de interfaces	Consultar no obliga a exponer métodos de guardar; repositorios se separan por agregado	DisponibilidadReader, PedidoReader, CotizacionRepository, PedidoRepository, InventarioRepository	UoW agrupa los colaboradores necesarios para coordinar, pero ubicacion_reserva pertenece al repositorio de inventario y no se duplica como operación del UoW.
D - inversión de dependencias	Servicios reciben fábrica de UoW y reloj; el detalle PostgreSQL se elige en la frontera de API	CotizacionService.__init__, PedidoService.__init__, application/ports.py, proveedores Depends	Las fábricas usan duck typing y no están anotadas con Protocol en cada firma. La API sí importa infraestructura porque es el punto de composición.

Un ejemplo de responsabilidad y sustitución

El endpoint de confirmación arma el comando y un actor de demo. El handler llama al servicio; el servicio invoca Pedido.confirmar y InventarioUbicacion.reservar. La misma operación de aplicación se usa en pruebas con un UoW Memory y en producción con PostgreSQL. Cambiar el transporte HTTP no obliga a reescribir las reglas; cambiar persistencia sí exige cumplir los contratos y verificar integridad/concurrencia del adaptador nuevo.

Un ejemplo de extensión de una regla

La vigencia no se calcula en el controller: CotizacionVigente.is_satisfied_by delega al Value Object. AceptacionDelCliente aporta otra condición. AndSpec combina ambas sin guardar pedidos, reservar inventario ni consultar PostgreSQL. La creación del pedido permanece en Cotizacion.convertir, y el commit en el servicio/UoW.

Evidencia y riesgos que el diseño no elimina

Comprobación	Evidencia existente
Condiciones puras y composición	tests/test_specifications.py
Dominio y rollback con Memory	tests/test_architecture.py, E9 y test_state_machine.py
Misma intención de API con persistencia real	tests/test_api_revalidacion_postgres.py, conversión, confirmación, cancelación y consulta PostgreSQL
Consultar sin escrituras	test_api_consulta_pedido_postgres.py::test_http_consulta_estado_persistido_sin_escrituras
Doble confirmación/cancelación y última unidad	Pruebas PostgreSQL de repetición/concurrencia

Hay colecciones públicas mutables en agregados y payload mutable dentro de EventoDominio congelado. La API usa actores y pago simulados; la auditoría no acredita permisos de usuario real. Los mensajes de ciertos rechazos se seleccionan comparando texto de excepciones, una dependencia frágil que debe revisarse si cambian los mensajes del dominio. Reconocer estos límites evita atribuir a la arquitectura garantías que el código aún no aporta.

Registros del 6 de noviembre de 2026. La implementación actual es la fuente para describir las decisiones vigentes. Los antecedentes U3 se mantienen intactos; su condición histórica evita interpretar Supabase, PostgreSQL pendiente o permisos no acreditados como características actuales.

Registro PIA	Decisión vigente	Antecedente
ADR-P01	Domain Model y servicios de aplicación	ADR001
ADR-P02	PostgreSQL local y en Render, con snapshots JSONB	ADR002
ADR-P03	Unit of Work, bloqueos y rollback del núcleo	ADR003
ADR-P04	CQRS básico con una sola base	ADR004
ADR-P05	Auditoría y outbox en la misma transacción	ADR005
ADR-P06	Specification combinable y máquina mediante enum/guardas	Nuevos componentes PIA
ADR-P07	Compose DB/App/Tests y despliegue después de CI	Nuevos componentes PIA

Cada registro contiene contexto, alternativas, decisión, consecuencias, evidencia y trazabilidad. Las decisiones delimitan el sistema existente; las historias de negocio todavía diseñadas conservan sus pendientes en el BRD.

Correcciones respecto a supuestos anteriores

Tema	Lectura vigente
Alojamiento	PostgreSQL local/Compose y Render; Supabase queda como alternativa histórica de U3.
Persistencia/concurrencia	Pruebas con PostgreSQL real y CI registradas; no se marcan como pendientes por textos antiguos.
Auditoría	Se registra transaccionalmente, con actor de demo; no se atribuyen permisos de BD ni consulta protegida que no se implementaron.
Estados	Enum y guardas de tres estados; no existe una clase State por estado ni surtido operativo.
CQRS	Separación básica en una base; revalidar/convertir siguen usando servicio directamente.
Despliegue	CI/CD inicialmente verificado en pia-render; la configuración final sigue main. El registro de cierre identifica las verificaciones efectivas.

La arquitectura, los diagramas, la aplicación de SOLID y la matriz arquitectural usan estas decisiones.

Los siete ADR completos están en docs/architecture/adrs/. Cada uno identifica contexto, alternativas, decisión, consecuencias y evidencia. La elección de PostgreSQL local/Render actualiza el supuesto histórico de Supabase sin modificar los documentos U3.

5. API y mensajes para el usuario

API 0.6.0 con seis operaciones. El OpenAPI versionado se contrastó con create_app().openapi() al actualizar la documentación técnica: coincide con el contrato generado por el código. Swagger utiliza el mismo OpenAPI en tiempo de ejecución.

Swagger público · **OpenAPI público** · **Swagger local**

Método y ruta	Caso y guía	Resultado válido
GET /inventario/{sku}	Disponibilidad, US01/US02	200: cantidades por ubicación registrada; sin escrituras.
POST /cotizaciones/{cotizacion_id}/versiones/{version}/revalidar	Revalidación, US04	201: siguiente versión vigente; conserva la anterior.
POST /cotizaciones/{cotizacion_id}/versiones/{version}/convertir	Conversión, US05	201: pedido Pendiente con origen/total y sin reserva.
POST /pedidos/{pedido_id}/confirmar	Confirmación, US06/US09	200: Confirmado y reserva en la misma transacción.
POST /pedidos/{pedido_id}/cancelar	Cancelación, parte de US10	200: Cancelado y reserva propia liberada.
GET /pedidos/{pedido_id}	Consulta	200: estado, folio/versión, total y moneda persistidos.

Presentación de Swagger

Swagger conserva sus operaciones y formularios. La vista sencilla presenta títulos de negocio y un mensaje comprensible por respuesta. **Mostrar detalles técnicos** recupera rutas, métodos y el JSON completo, incluido el ID del pedido necesario para continuar el recorrido. El contrato HTTP y los datos persistidos no cambian al alternar esta vista. Server response muestra una ejecución real; Example Value es un ejemplo de documentación.

Entradas y errores

Los importes/cantidades usan representación decimal; las solicitudes rechazan tipos/campos inválidos y campos extra según su modelo Pydantic. El cliente no decide estado, total ni cantidades a reservar al confirmar: el servidor los recupera del pedido guardado. Cancelar recibe motivo, no unidades que el cliente pretenda liberar.

HTTP	Uso en el contrato
400	SKU vacío después de recortar espacios, cuando la ruta sí contiene un valor.
404	Identificador válido inexistente o SKU no encontrado en inventario.

HTTP	Uso en el contrato
409	Vigencia, aceptación, estado, pago o disponibilidad impiden el comando; se conserva el estado previo durable.
422	Datos/UUID/versiones/campos inválidos o problemas de ubicación/partidas según la operación.
503	Conexión o persistencia temporalmente no disponible; mensaje sin detalles internos.

El cuerpo comunica mensaje en español y, cuando corresponde, el identificador. Abrir /docs o /openapi.json no abre por sí mismo una conexión de datos; para comprobar PostgreSQL se ejecuta además una consulta de inventario o pedido.

Datos de demostración y repetición del recorrido

La cotización ficticia base tiene ID 11111111-1111-4111-8111-111111111111, versión 1 vencida. La semilla incluye inventario ficticio y conserva registros existentes. En una base nueva, la guía de revalidación crea versión 2. En una base que ya contiene esa versión, no debe repetirse la operación suponiendo que el historial esté vacío: primero se identifica la versión/estado real. La vigencia es temporal; una versión creada durante la práctica puede vencer después.

Cada conversión válida crea un pedido nuevo. El ID se toma de la respuesta, no se sustituye por el ID de la cotización ni por el UUID ilustrativo de las guías. Confirmación/cancelación repetidas se rechazan sin duplicar efectos; esto no vuelve idempotente a la conversión. El script HTTP de CI está preparado para una base ficticia nueva del runner y no se utiliza para repetir el demo sobre la base con historial.

La evidencia pública registrada conserva el pedido 4d7b5b3e-926e-4856-9a1e-d1a433ef5cfd, Cancelado, origen versión 2 y total 240.00 MXN. Sus consultas permiten comprobar datos sin crear otro pedido. El despliegue distingue datos locales y públicos.

Alcance y trazabilidad

Los actores Vendedor/Caja/Cancelación son fijos de demo y el pago es una referencia/booleano ficticios. No existe autenticación de personas, pasarela, reembolso, surtido operativo ni consulta de auditoría en estos endpoints. Las condiciones comerciales y pendientes siguen en el BRD.

Las pruebas HTTP con Memory y PostgreSQL identificadas en trazabilidad de análisis y arquitectura comprueban contrato, reglas y persistencia. El archivo Gherkin expresa criterios del análisis; no se presenta como suite BDD ejecutada.

5.1 Interpretación de respuestas

Swagger mantiene sus formularios y apariencia. La vista sencilla muestra títulos de negocio y mensajes comprensibles; la opción Mostrar detalles técnicos recupera rutas, métodos e identificadores. Ese control cambia la presentación y conserva el contrato HTTP completo. Server response es la respuesta de una petición ejecutada; Example Value es un ejemplo del contrato.

La creación informa que el pedido sigue pendiente y no tiene productos reservados. La confirmación exige pago de demo autorizado y existencias; un rechazo conserva el estado durable. La cancelación solo admite

un pedido confirmado y todavía no surtido.

6. Pruebas y evidencias verificables

Evidencia	Resultado y procedencia
Suite completa registrada	249 aprobadas; 0 errores, 0 fallos, 0 omitidas; 97.38% de cobertura.
Código de esa ejecución	c4876efe91a3d3b2a801017e54017718a593db61.
Ejecución GitHub Actions	42322301835; Docker, recorrido HTTP, navegador Swagger y pytest con PostgreSQL.
Cobertura	818 de 840 líneas ejecutables cubiertas; 22 faltantes. Supera el mínimo 80%.
Reporte de resultados	evidencias/pia/ci_swagger_verificacion.json; artefactos JUnit y coverage.json de la ejecución.
Alcance de la métrica	Cobertura de líneas del núcleo probado. No certifica todas las historias ni la seguridad de una operación real.

Ejecución y artefactos: https://github.com/Santana-R/ferremax-u3/actions/runs/42322301835

Las pruebas de integración utilizan una base temporal propia y datos ficticios. El runner conserva reportes y elimina esa base; no elimina la base del demo. Las comprobaciones incluyen primera escritura seguida de fallo, doble confirmación/cancelación, consulta sin escrituras y concurrencia por última unidad.

6.1 Capturas de ejecuciones reales de Swagger

Las siguientes capturas provienen del navegador de CI con App/DB reales y datos ficticios de la ejecución citada. No se atribuyen a la base pública de Render. Los resultados ejecutados se identifican por Server response; debajo aparecen ejemplos estáticos que no cuentan como pruebas.

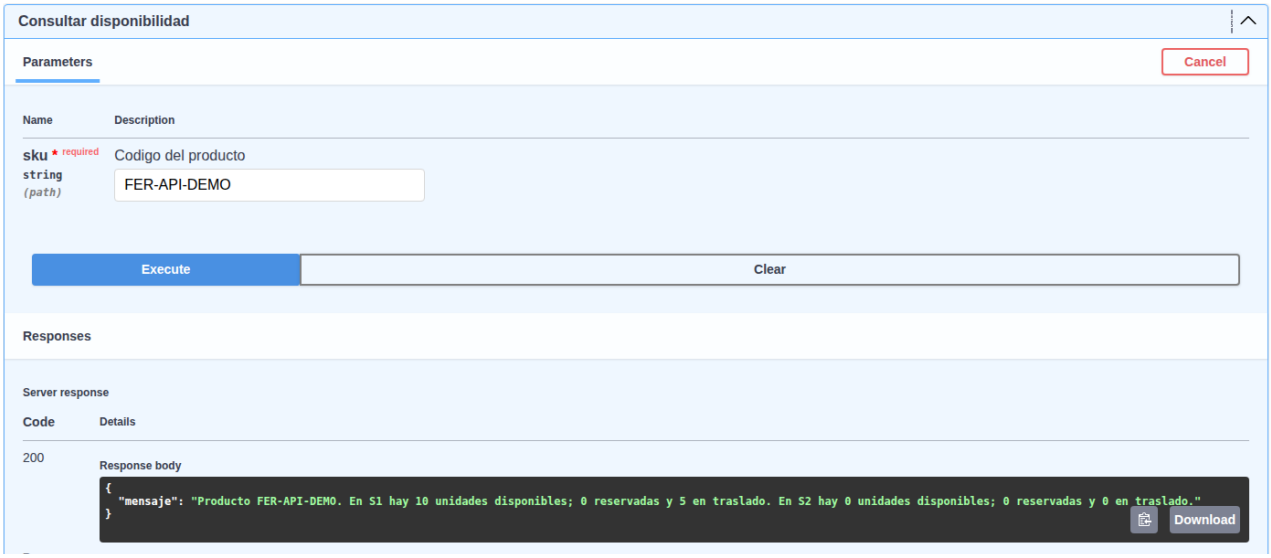


Figura E1. GET ejecutado, HTTP 200: FER-API-DEMO; S1 informa 10 disponibles, 0 reservadas y 5 en traslado; S2 informa ceros. Encuadre nativo del documento; el PNG original se conserva completo en los artefactos CI.

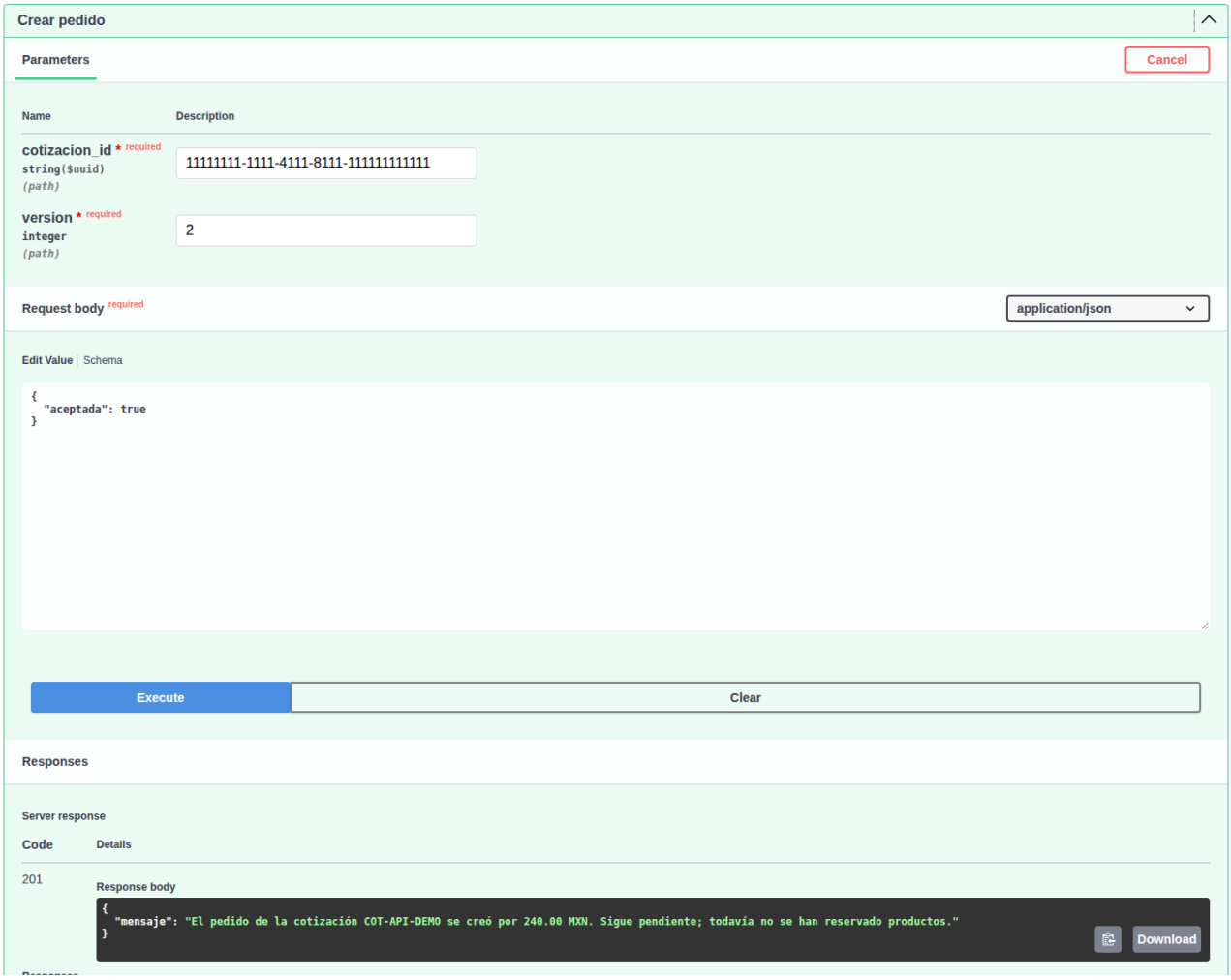


Figura E2. Creación aceptada HTTP 201 por 240.00 MXN; pedido Pendiente sin reserva. Encuadre de la entrada y Server response; no incluye los ejemplos estáticos.

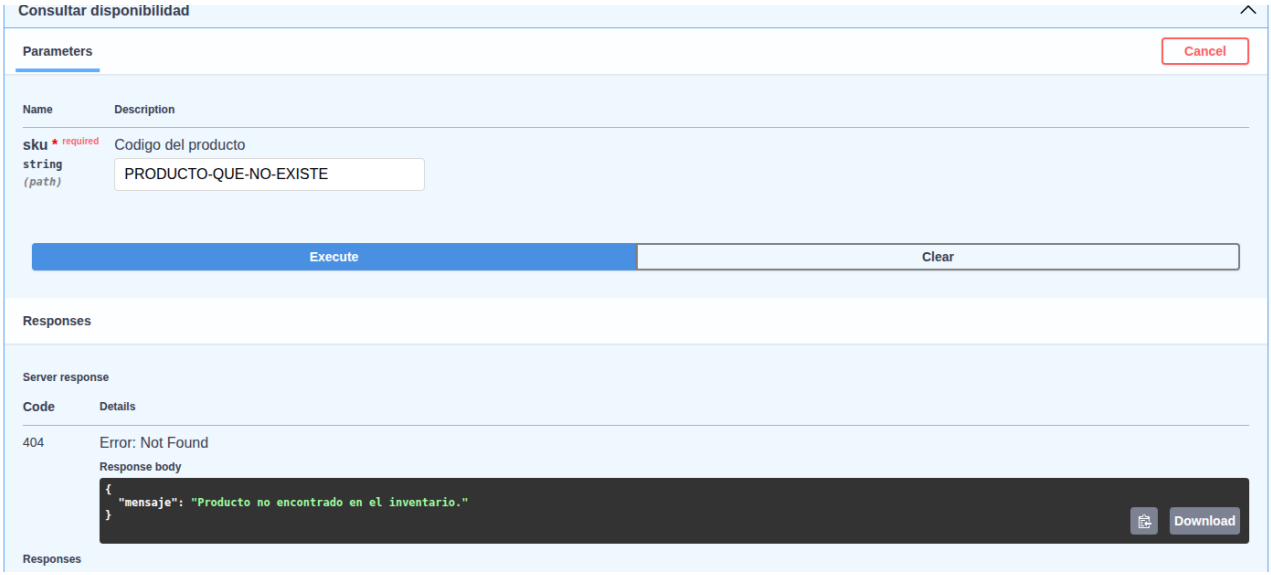


Figura E3. Consulta ejecutada con PRODUCTO-QUE-NO-EXISTE y HTTP 404. Encuadre de la misma captura original; los resultados no se editan.

Transcripción de los resultados reales: “El pedido de la cotización COT-API-DEMO se creó por 240.00 MXN. Sigue pendiente: todavía no se han reservado productos.” y “Producto no encontrado en el inventario.”

6.2 Recorrido público registrado

Operación registrada	Respuesta y efecto
Cotización vencida / sin aceptación	409; no crea el pedido.
Revalidar / convertir versión aceptada	201; conserva versión anterior y crea Pendiente por 240.00 MXN.
Pago rechazado	409; pedido e inventario se conservan.
Confirmar con autorización simulada	200; Confirmado, 2 comprometidas y 8 disponibles.
Repetir confirmación	409; no duplica reservas.
Cancelar antes de surtir	200; Cancelado, 0 comprometidas y 10 disponibles.
Repetir cancelación	409; no libera dos veces.

Procedencia: evidencias/pia/render_verificacion.json y render_cd_verificacion.json. Pedido público registrado: 4d7b5b3e-926e-4856-9a1e-d1a433ef5cfd. Estos archivos identifican ejecuciones anteriores y no se presentan como una nueva ejecución de cierre.

7. Docker, CI/CD y despliegue

Docker Compose contiene db, app y tests. DB conserva el volumen de demostración; App utiliza PostgreSQL por red interna de Compose; Tests crea una base temporal distinta. Configuración local se define en .env y la plantilla .env.example no contiene secretos reales. La API escucha el puerto publicado; en Render se utiliza PORT y DATABASE_URL interna.

```
docker compose config --quiet
docker compose up -d --build --wait app
docker compose run --build --rm tests
docker compose stop app db
```

El pipeline construye imágenes, levanta App/DB, comprueba las seis operaciones por HTTP, verifica la presentación de Swagger en un navegador y ejecuta pytest. El umbral de cobertura es 80% y no se admiten omisiones. Los resultados JUnit/cobertura se conservan como artefactos vinculados al commit.

La configuración final sigue main con autoDeployTrigger: checksPass. Render publica el servicio Docker y mantiene PostgreSQL separado por red interna. El arranque de esquema/semilla es idempotente: no borra pedidos ni reinicia cotizaciones existentes. El registro de cierre confirma el deploy efectivo después de la integración.

El plan Free puede dormir por inactividad y despertar con la siguiente petición. La base gratuita tiene caducidad registrada en diciembre; la demostración debe comprobarse antes de la defensa. /openapi.json acredita el servidor; una consulta de inventario/pedido acredita además acceso a datos.

8. Conservación de U3 y entrega

La etiqueta u3-entrega apunta al commit eb71b66 y conserva la entrega evaluada de U3. Main recibe el PIA manteniendo el historial. Los originales U2/U3, diagramas, ADRs y PDF históricos no se reemplazan. Las ramas de etapas se conservan; no hay que fusionarlas una por una.

El equipo debe revisar estos documentos, cargar los entregables en NetAcad y demostrar el mismo repositorio/despliegue. Este informe no acredita una carga en plataforma ni una calificación. La defensa requiere explicar reglas, transacciones y límites, además de ejecutar la demostración.

9. Conclusiones

Se integró un núcleo de negocio FerreMax que distingue elegibilidad de ejecución: una cotización apta todavía no es un pedido confirmado, y crear un pedido no reserva inventario. La confirmación y cancelación coordinan estado, existencias y registros dentro de una misma transacción.

Specification permite componer reglas puras; los servicios de aplicación coordinan persistencia mediante Unit of Work; la máquina de estados limita operaciones. Las pruebas con PostgreSQL y los rechazos repetidos sustentan la integridad del núcleo. Los mensajes de Swagger permiten explicar al usuario qué ocurrió sin exigirle interpretar el JSON técnico.

La implementación todavía no materializa todo el mapa de negocio de U2. La cobertura 97.38% describe las líneas de código probadas y no elimina esos pendientes. La siguiente evolución debe priorizar autenticación, pagos reales y los procesos de negocio faltantes, conservando trazabilidad y pruebas de aceptación.

Bibliografía y fuentes

Gómez Marroquín, M. Á. (2026). PIA CLN: Proyecto Integrador AD2026 [Indicaciones de clase, pp. 1–4]. UANL–FIME.

Equipo 17. (2026). FerreMax: BRD, historias, BPMN y glosario de U2; informe y arquitectura U3. Archivos originales en reference/, diagrams/ y docs/ del repositorio.

Equipo 17. (2026). FerreMax PIA: análisis integrado, modelo DDD, ADR, código y pruebas. https://github.com/Santana-R/ferremax-u3

GitHub Actions. (2026). FerreMax CI, ejecución 42322301835 [JUnit, cobertura y capturas de Swagger]. https://github.com/Santana-R/ferremax-u3/actions/runs/42322301835

Render. (2026). Deploys; Health checks; Blueprint YAML reference. https://render.com/docs/deploys ; https://render.com/docs/health-checks ; https://render.com/docs/blueprint-spec

Equipo 17. (2026). U4 Informe: TechStore [Referencia de formato de reporte]. Documento Word proporcionado por Roberto.

Anexo A. BRD integrado completo

Dato	Valor
Curso	Componentes de Lógica de Negocios
Profesor	Miguel Ángel Gómez Marroquín
Equipo	17
Integrantes	Roberto Santana Trujillo, 2227084; Isaías Leonardo Salazar Torres, 2227016
Fecha de integración	6 de noviembre de 2026
Base	BRD, historias/Gherkin, BPMN y glosario de U2; arquitectura de U3; implementación PIA

1. Propósito y visión

FerreMax necesita coordinar cotizaciones, pedidos e inventario para ofrecer mercancía disponible, conservar las condiciones aceptadas por el cliente y registrar las decisiones que afectan una venta. El análisis abarca también crédito, transferencias, parciales, surtido, entregas, devoluciones y control operativo.

Este BRD integra los documentos de U2/U3 y los relaciona con la solución actual. Mantiene las necesidades del negocio aunque una función todavía no esté implementada. La trazabilidad por criterio distingue lo implementado, simulado, parcial y diseñado; documentar una limitación no elimina el requisito ni acredita por sí mismo el cumplimiento del PIA.

2. Contexto del negocio y operación actual

FerreMax vende herramientas y materiales eléctricos, de plomería y construcción. Opera tres sucursales y un almacén central; atiende mostrador y clientes empresariales que pueden solicitar crédito. Sus productos se identifican por SKU y se controlan por ubicación.

En la operación descrita en U2, el vendedor consulta existencias y ofrece surtido local, desde almacén central o mediante otras sucursales. Las cotizaciones se manejan en hojas de cálculo, con riesgo de aplicar condiciones vencidas o versiones distintas. El crédito requiere revisar la cuenta y separar una autorización normal de una excepción. Los pedidos grandes pueden necesitar unidad, operador y evidencia de entrega.

Problema identificado	Consecuencia	Necesidades relacionadas
Prometer mercancía ya comprometida	Ventas que no pueden surtirse, retrasos y cancelaciones	BN01/BN02/BN04
Cotizaciones dispersas o vencidas	Precios, condiciones y versiones inconsistentes	BN03
Visibilidad limitada de otras ubicaciones y tránsito	Alternativas de surtido sin respaldo	BN01/BN06/BN07

Problema identificado	Consecuencia	Necesidades relacionadas
Crédito excepcional sin decisión trazable	Riesgo comercial y falta de responsables	BN05/BN10
Entregas y devoluciones sin evidencia suficiente	Incidencias y ajustes difíciles de explicar	BN08/BN09/BN10

3. Objetivos y forma de medición

Los objetivos son metas del negocio heredadas de U2, con acuerdos simulados para el ejercicio académico. No se presentan como resultados operativos obtenidos.

ID	Objetivo conservado	Medición y estado
OB01	Reducir al menos 80% las ventas confirmadas sin mercancía durante los primeros tres meses frente a la línea base	Comparar incidencias del periodo con una línea base depurada. No hay medición de operación real; el núcleo sí prueba rechazo de sobreventa.
OB02	Conseguir que el 100% de cotizaciones tengan folio, versión, estado y vigencia; usar siete días como valor de referencia del caso	Comprobar campos y vigencia configurada. La emisión operativa inicial está pendiente; revalidar recibe el número de días, no impone siete.
OB03	Lograr que el 100% de autorizaciones excepcionales de crédito sean trazables	Revisar cada decisión con responsable, fecha, motivo y condición. Flujo pendiente.
OB04	Consultar disponibilidad global en <=3 s para al menos 95% de consultas normales	Ensayo de rendimiento con carga y condiciones declaradas. Aún sin medición.
OB05	Mantener estado y responsables visibles en todas las transferencias y entregas activas	Conciliar movimientos y evidencia por folio. Flujos pendientes.

En U2, CE01 prometía eliminación total de ventas sin disponibilidad mientras OB01 fijaba una reducción de 80%; CE04 omitía el percentil de OB04. Esta integración alinea los criterios operativos a los objetivos medibles de U2. La prohibición de sobrecomprometer inventario se conserva como regla de integridad del núcleo, distinta del indicador operativo de tres meses.

4. Stakeholders y responsabilidades

Actor	Responsabilidad en el negocio	Situación en la aplicación
Cliente	Solicita productos, acepta una versión de cotización, decide parciales, paga y recibe	La aceptación y el pago se proporcionan como datos de demostración; no hay portal de cliente.
Vendedor	Consulta disponibilidad, emite/revalida cotizaciones, crea pedidos y propone alternativas	Consulta, revalidación y conversión implementadas con actor simulado; emisión inicial y alternativas pendientes.
Caja/Crédito	Registra pagos, evalúa crédito y resuelve excepciones	Confirmación de contado con autorización simulada; crédito integral pendiente.
Almacén	Controla inventario, surte, transfiere y recibe	Reserva/liberación automáticas del núcleo; surtido y transferencias pendientes.
Compras	Atiende bajo mínimo y reabastecimiento	Participación informativa del análisis; integración con proveedores fuera del alcance original.
Coordinador y operador de entrega	Programan, trasladan y capturan evidencia o incidencia	Diseñado.
Gerente de operaciones	Define prioridades y consulta indicadores	Panel diseñado; objetivos operativos sin medir.
Auditor/administrador	Consulta cambios y administra parámetros autorizados	Se registra auditoría de los casos implementados; consulta protegida y permisos nominales pendientes.

Los roles del código son guardas de servicios. Los endpoints construyen actores ficticios: no acreditan autenticación de personas ni autorización real de un usuario conectado.

5. Alcance del negocio y alcance ejecutable actual

5.1 Necesidades conservadas

Se conserva el alcance de U2: disponibilidad global y reservas; cotizaciones, venta y crédito; transferencias, parciales y surtido; entregas y devoluciones; alertas y auditoría. Se mantienen las veinte historias y sus criterios en USER_STORIES.md.

Permanecen fuera del alcance original: comercio electrónico público general, contabilidad general y nóminas, optimización de rutas e integración con proveedores. Esta lista no incluye funciones pendientes que sí pertenecían a U2.

5.2 Recorrido que se puede ejecutar y demostrar

Operación	Regla y resultado	Historias
GET /inventario/{sku}	Consulta por ubicación física, comprometida, tránsito y disponible; no modifica datos	US01/US02
POST /cotizaciones/{id}/versiones/{version}/revalidar	Revalida una versión vencida y conserva su historial; no reserva	US04
POST /cotizaciones/{id}/versiones/{version}/convertir	Exige vigencia y aceptación; crea pedido Pendiente ligado a la versión; no reserva	US05
POST /pedidos/{id}/confirmar	Exige autorización de pago y existencias; guarda Confirmado y reserva en una transacción	US06/US09
POST /pedidos/{id}/cancelar	Exige Confirmado, aún no surtido y motivo; libera la reserva propia y guarda Cancelado	Parte de US10
GET /pedidos/{id}	Recupera el estado, folio/versión de origen, total y moneda persistidos	Apoya US05/US06/US09/US10

5.3 Límites concretos de ese recorrido

• La confirmación ejecutable es de contado, con un booleano y una referencia que simulan autorización; no procesa dinero ni ejecuta crédito.

• La reserva utiliza una sola ubicación por pedido. Revalidación y conversión consultan S1, fijada en el servicio. La consulta global devuelve las ubicaciones que realmente tienen un registro para el SKU; el demo público no completa necesariamente las cuatro ubicaciones del negocio.

• La emisión inicial de cotizaciones se prepara mediante semilla ficticia; no existe un endpoint de emisión ni de consulta independiente de cotización.

• Cancelar está implementado; reducir cantidades aún no. El indicador surtido permite probar una guarda, pero no existe una operación para surtir ni estados Entregado/Devuelto en la máquina actual.

• Crédito tiene una política básica aislada; faltan solicitud, autorización excepcional, persistencia e integración. Transferencias, parciales, surtido, entrega, devolución y panel siguen diseñados.

• La auditoría del núcleo guarda actor, fecha, antes/después, acción y motivo; la identidad es ficticia. No hay API de consulta de bitácora ni evidencia de permisos de BD que impidan edición a un usuario operativo.

U3 detalló cinco casos de uso y la aplicación añadió una consulta de pedido. No existe evidencia en este repositorio de una dispensa del profesor para las demás historias del PIA. Por ello las limitaciones se mantienen como trabajo pendiente de cumplimiento, con su trazabilidad visible.

6. Necesidades del negocio

ID	Necesidad	Prioridad U2	Historias	Estado actual
BN01	Disponibilidad confiable de las tres sucursales y almacén	Alta	US01/US02	Consulta implementada sobre registros disponibles; objetivo temporal sin medir.
BN02	Reservar y liberar sin prometer mercancía comprometida	Alta	US09/US10	Reserva y cancelación implementadas; reducción pendiente.
BN03	Cotizaciones controladas y versionadas	Alta	US03/US04/US05	Revalidación/conversión implementadas; emisión inicial operativa pendiente.
BN04	Venta y surtido local, multiorigen o parcial con consentimiento	Alta	US05/US06/US11/US14/US15	Núcleo de contado local; resto pendiente.
BN05	Crédito empresarial normal y excepcional	Alta	US07/US08	Evaluación básica en dominio; flujo integral pendiente.
BN06	Transferencias con responsables, cantidades y estado	Alta	US11/US12/US13	Diseñado.
BN07	Cantidades surtidas y pendientes por partida	Media	US14/US15	Diseñado.
BN08	Entrega programada con receptor y evidencia o incidencia	Media	US16/US17	Diseñado.
BN09	Devoluciones según comprobante, categoría y condición	Media	US18	Diseñado; política comercial aún por precisar.
BN10	Alertas, reportes y auditoría	Alta	US19/US20	Registro transaccional en el núcleo; panel y consulta protegida pendientes.

7. Reglas de negocio

ID	Regla conservada y precisión PIA	Historias	Estado
BR01	Una cotización y su revalidación/conversión no reservan inventario	US03/US04/US05	Verificada en revalidación/conversión; emisión operativa pendiente.
BR02	Cotización con folio, versión, vigencia y estado. Vigente cuando emisión <= instante < vencimiento	US03/US04/US05	Modelo/versionado implementados; emisión operativa pendiente.
BR03	Una versión vencida se revalida antes de convertirse; la nueva conserva la anterior	US04/US05	Implementada.
BR04	Disponible = existencia física - comprometida, sobre movimientos confirmados. Tránsito no se suma	US01/US02/US09	Implementada para inventario y reservas del núcleo.
BR05	Solo un pedido Confirmado puede crear reserva; estado y reserva se confirman juntos después de autorización y validaciones	US06/US09/US15	Implementada en contado local.
BR06	Cancelar o reducir antes del surtido libera la cantidad propia que corresponde, con autorización y motivo	US10	Cancelación completa implementada; reducción pendiente.
BR07	Ante faltante local se consulta primero almacén central y después otras sucursales	US11/US15	Diseñada; no hay selección automática multiorigen.
BR08	Un parcial exige aceptación explícita de cantidades surtibles/pendientes	US14	Diseñada; aceptar una cotización no basta.
BR09	Solo clientes empresariales activos pueden solicitar crédito	US07	Política básica parcial; sin flujo ni registro empresarial completo.
BR10	Crédito normal no excede disponible ni ignora bloqueo; importes deben tener moneda compatible	US07	Evaluación aislada parcial; sin autorización integral.

ID	Regla conservada y precisión PIA	Historias	Estado
BR11	Toda excepción de crédito registra decisión, responsable, fecha, condición y motivo de Caja/Crédito	US08	Diseñada.
BR12	Transferencia con origen, destino, partidas, responsable y estado	US11/US12	Diseñada.
BR13	Recepción de transferencia actualiza tránsito/inventario de destino conservando diferencias como incidencias	US13	Diseñada.
BR14	Entrega con receptor, fecha/hora y evidencia; una incidencia no cierra el pedido como entregado	US16/US17	Diseñada.
BR15	Devolución según política configurable por categoría y condición del producto	US18	Diseñada; valores de política pendientes.
BR16	Toda modificación sensible registra actor, fecha/hora, acción, valores anteriores/nuevos y motivo	US19/US20 y operaciones sensibles	Registro implementado en el núcleo; consulta, permisos e identidad real pendientes.

Atomicidad y estados: si falta stock durante una confirmación, se revierte toda la transacción y el pedido persistido permanece Pendiente. Esto no añade una transición comercial Confirmado → Pendiente. Repetir confirmación o cancelación se rechaza sin duplicar efectos. Con tres disponibles, dos o exactamente tres son válidos; cuatro no.

8. Requisitos no funcionales

ID	Requisito	Evidencia o condición pendiente
RNF01	Consulta global <=3 s en 95% de consultas normales; reserva <=5 s	Se requiere ensayo declarado de carga, entorno y percentiles. Aún sin medir.
RNF02	La última unidad no se asigna a dos pedidos concurrentes	Pruebas con PostgreSQL real, bloqueos y rollback; también concurrencia de revalidación/cancelación.
RNF03	Acceso por rol y autorización separada para crédito, cancelación y devolución	Guardas de servicio verificadas con actores ficticios; autenticación y permisos operativos pendientes.
RNF04	Historial de cambios con quién, cuándo, qué y por qué	Auditoría transaccional en el núcleo; identidad real y consulta protegida pendientes.
RNF05	Operación durante el horario de las cuatro ubicaciones	Nivel exacto de servicio y medición pendientes; Render Free puede suspender por inactividad.
RNF06	Estados, causas y siguientes acciones en lenguaje del negocio	Mensajes HTTP en español verificados para el núcleo. No hay estudio de usabilidad de usuarios.
RNF07	No confirmar pedidos, transferencias ni entregas con datos obligatorios incompletos	Datos y transacciones del pedido verificados; los otros procesos siguen pendientes.
RNF08	Vigencias, políticas y umbrales comerciales no confirmados parametrizados	Días de revalidación son parámetro; políticas completas de crédito/devolución pendientes. Conexión y arranque usan variables de entorno.

9. Proceso propuesto y correspondencia con BPMN

El BPMN integrado tiene siete vistas relacionadas: cotización/pedido; confirmación/cancelación; crédito; transferencias/parciales; surtido/entrega; devoluciones; auditoría/reportes. Incluye el proceso del negocio completo y marca cada actividad según su soporte actual. Es un modelo BPMN descriptivo, no un motor de ejecución.

La vista de contado se puede seguir en Swagger: consultar disponibilidad, revalidar una cotización vencida, registrar aceptación y crear Pendiente, autorizar el pago de demo, confirmar/reservar y, antes del surtido, cancelar/liberar. La consulta del pedido verifica el estado persistido entre operaciones.

Se corrige la secuencia del diagrama U2 que colocaba reserva antes del pago. En PIA, una cotización no reserva y un pago rechazado deja el pedido Pendiente. El fallo de stock tampoco crea una reserva parcial. La cancelación requiere Confirmado y aún no surtido.

10. Datos y lenguaje del dominio

El glosario integrado conserva los términos U2 y precisa cotización, versión, vigencia, pedido Pendiente, autorización simulada, reserva propia, tránsito, auditoría, Specification y transacción.

Los importes utilizan Decimal y moneda; cantidades deben ser positivas y finitas. La cotización identifica al cliente y conserva partidas numeradas; el pedido mantiene folio/versión de origen y su propio total. Las fechas incluyen zona horaria y los instantes técnicos se registran en UTC. La comparación de vigencia incluye emisión y excluye vencimiento.

11. Supuestos, restricciones y cuestiones abiertas

Tema	Tratamiento
Caso académico	Los acuerdos de siete días y metas operativas son simulados; no se atribuyen entrevistas nuevas ni resultados reales a FerreMax.
Datos maestros	Son ficticios; las cuatro ubicaciones del negocio no implican registros de cada SKU en todas ellas. No existe catálogo maestro independiente en la API.
Precios y condiciones	El vendedor aporta datos al revalidar; no hay proveedor externo de precios.
Crédito	Deben definirse responsables nominales, condiciones y límites de excepción antes de implementar el flujo completo.
Devoluciones	Faltan política por categoría/condición, plazos y decisiones financieras. No se inventan valores.
Identidad y pago	Simulados en la demostración. La validación interna de un rol no equivale a autenticación externa.
Infraestructura	PostgreSQL local/Compose y servicio público en Render; la evidencia debe distinguir ejecuciones y commits.
Entrega PIA	La integración documental conserva todos los requisitos. Las funciones diseñadas siguen pendientes; cualquier recorte deberá constar como decisión explícita, no inferirse de la revisión U3.

12. Criterios de éxito

ID	Criterio refinado	Comprobación actual
CE01	Cumplir OB01: reducir >=80% las incidencias de venta sin mercancía en tres meses frente a la línea base	Sin medición operativa. La suite prueba rechazo de sobreventa/rollback en el núcleo.
CE02	Cada cotización tiene folio, versión, vigencia y estado; la vigencia de referencia de siete días puede configurarse	Revalidación/versionado verificados; emisión inicial operativa pendiente.
CE03	Toda decisión de crédito, normal o excepcional, conserva trazabilidad suficiente	Pendiente en el flujo integral.
CE04	Cumplir OB04 y RNF01: <=3 s en al menos 95% de consultas normales	Pendiente de medición.
CE05	Transferencias y entregas activas tienen estados y responsables consultables	Diseñado.
CE06	Devoluciones se resuelven según política de categoría/condición y auditoría	Diseñado; política pendiente.
CE07	Entregas conservan operador, receptor y evidencia o incidencia; una incidencia no se cuenta como entrega	Diseñado.
CE08	Transferencias concilian origen, tránsito, recepción y diferencias	Diseñado.

13. Trazabilidad y aceptación verificable

• Veinte historias y criterios, con identificadores históricos y correcciones explícitas.

• Contrato Gherkin: cuarenta y seis escenarios de aceptación, sin presentarlos como pruebas BDD ejecutadas.

• Matriz por criterio: necesidad, regla, actividad BPMN, componente, prueba y límite.

• Matriz de los requisitos del profesor: diagnóstico general del cierre.

La ejecución registrada para 947aeae obtuvo 242 pruebas aprobadas, 0 omitidas y 97.24% de cobertura. La evidencia incluye CI, despliegue automático y conservación de datos. Es una medida de las líneas y casos ejecutados de esa versión, no de todas las necesidades del negocio ni del cumplimiento total de las veinte historias.

14. Control de cambios y fuentes

Refinamiento PIA	Razón
Unificar AC y corregir el identificador repetido de US01	Mantener trazabilidad inequívoca.
Separar cancelación y reducción de US10	Mostrar la diferencia real de implementación sin eliminar la operación pendiente.
Precisar vigencia, reserva y rollback	Expresar los límites que verifican código y pruebas.
Alinear CE01/CE04 con OB01/OB04	Evitar metas contradictorias dentro del BRD.
Corregir orden de pago/confirmación/reserva en BPMN	Respetar BR01/BR05 y el comportamiento transaccional.
Conservar alcance amplio y señalar pendientes	Evitar presentar una vista de diseño como una aplicación completa.

Fuentes: BRD U2.pdf>), historias U2.pdf>), BPMN U2.pdf>), glosario U2.pdf>), informe U3 y código/pruebas de la base d1eebde. Los archivos históricos y la etiqueta u3-entrega se conservan sin cambios.

Anexo B. Veinte historias y criterios de aceptación

Se conservan los criterios de todas las historias, incluso donde el flujo está diseñado o parcialmente implementado. Dado/Cuando/Entonces corresponde al contrato Gherkin de análisis. Las pruebas pytest ejecutadas están identificadas por sus archivos; el archivo .feature no se declara ejecutado como suite.

Versión integrada del 6 de noviembre de 2026. Equipo 17: Roberto Santana Trujillo e Isaías Leonardo Salazar Torres.

Se conservan **US01-US20** de U2. Los criterios siguientes precisan el comportamiento y mantienen los escenarios no implementados como requisitos. El BRD define el alcance del negocio y sus límites de implementación; el BPMN usa los mismos identificadores.

Los escenarios completos están también en aceptacion.feature. Ese archivo es un contrato Gherkin del análisis, no una suite ejecutada: los resultados reales provienen de las pruebas pytest enlazadas en trazabilidad_analisis.csv.

Estado	Significado
Implementada	Hay comportamiento y prueba identificable para el criterio señalado.
Simulada	El flujo se verifica con una entrada o actor ficticio; falta la integración real.
Parcial	Solo una parte de la historia o criterio tiene soporte.
Diseñada	Requisito conservado sin flujo ejecutable completo.

Correspondencia de identificadores históricos

Identificador U2	Identificador PIA	Motivo
AC-US01-01, consulta global	AC-US01-01	Se mantiene.
AC-US01-01, SKU inexistente (segundo uso)	AC-US01-02	Se elimina duplicidad usando el título para desambiguar.
AC-US10-01, cancelar o reducir	AC-US10-01 cancelación; AC-US10-02 reducción	Se separan dos operaciones con distinto estado de implementación.
CA-US18-01 / CA-US19-01 / CA-US20-01	AC-US18-01 / AC-US19-01 / AC-US20-01	Se normaliza el prefijo, conservando contenido y relación.
Sin ID en U2	Nuevos AC descritos como refinamiento PIA	Explicitan límites, concurrencia y ausencia de efectos parciales.

US01 - Consultar la disponibilidad global de un producto por sku

Como vendedor, quiero consultar la disponibilidad global de un producto por SKU, para ofrecer una cantidad que realmente pueda surtirse.

Necesidades: BN01. **Reglas:** BR04. **Estado de la historia:** Implementada.

BPMN: P01_T01. **Componente:** src/ferremax/api/main.py; src/ferremax/application/cqrs.py; src/ferremax/infrastructure/postgres.py.

Límite: Se devuelven las ubicaciones registradas para el SKU, sin inventar existencias ni sumar tránsito al disponible. La semilla pública tiene menos registros que las cuatro ubicaciones del negocio.

Criterio	Dado	Cuando	Entonces	Estado
AC-US01-01 - Consulta global	un SKU con inventario registrado por ubicación	el vendedor consulta su disponibilidad	se muestran existencia física, comprometida, en tránsito y disponible de cada ubicación registrada	Implementada
AC-US01-02 - SKU inexistente	un SKU que no tiene registros en el inventario consultable	el vendedor consulta su disponibilidad	se informa que no fue encontrado y no se inventan existencias	Implementada

Pruebas y límites de los criterios:

• AC-US01-01: tests/test_api_postgres.py, método test_consulta_http_lee_postgres_sin_modificar_inventario.

• AC-US01-02: tests/test_api_postgres.py, método test_consulta_http_producto_no_existente_en_postgres. Se corrige el ID repetido sin cambiar el escenario. No hay catálogo maestro independiente del inventario en la API.

US02 - Distinguir inventario físico, comprometido y disponible

Como vendedor, quiero distinguir inventario físico, comprometido y disponible, para no confundir mercancía presente con mercancía vendible.

Necesidades: BN01/BN02. **Reglas:** BR04. **Estado de la historia:** Implementada.

BPMN: P01_T01. **Componente:** src/ferremax/domain/model.py; src/ferremax/infrastructure/postgres.py.

Límite: El tránsito se informa por separado; una consulta no reserva ni modifica datos.

Criterio	Dado	Cuando	Entonces	Estado
AC-US02-01 - Cálculo de disponible	un SKU con 10 unidades físicas y 7 comprometidas en una ubicación	se consulta esa ubicación	el disponible mostrado es 3 unidades	Implementada
AC-US02-02 - El tránsito no es vendible	un SKU con 0 unidades disponibles y 5 en tránsito	se consulta su disponibilidad	el disponible permanece en 0 y las 5 unidades se muestran como tránsito	Implementada

Pruebas y límites de los criterios:

• AC-US02-01: tests/test_e9_inventario.py, método InventarioE9Test::test_e9_inv_01_exito_reservar_dos_deja_una_disponible.

• AC-US02-02: tests/test_api_inventario.py, método test_producto_sin_disponible_existe_y_transito_no_se_suma.

US03 - Crear una cotización con vigencia y condiciones comerciales

Como vendedor, quiero crear una cotización con vigencia y condiciones comerciales, para entregar una propuesta controlada al cliente.

Necesidades: BN03. **Reglas:** BR01/BR02. **Estado de la historia:** Parcial.

BPMN: P01_T02. **Componente:** src/ferremax/domain/model.py; scripts/cargar_cotizacion_demo.py.

Límite: Hay entidad y carga de datos ficticios; falta un caso de uso y endpoint de emisión inicial por vendedor.

Criterio	Dado	Cuando	Entonces	Estado
AC-US03-01 - Emisión inicial	un cliente y partidas válidas con precios y condiciones	el vendedor emite una cotización	se asignan folio, versión, emisión, vencimiento, totales y estado vigente, se conserva la disponibilidad consultada y no se reserva inventario	Diseñada

Pruebas y límites de los criterios:

• AC-US03-01: sin prueba integral identificada. La semilla de demostración no acredita una emisión operativa ni almacenamiento de una consulta de disponibilidad.

US04 - Revalidar una cotización vencida

Como vendedor, quiero revalidar una cotización vencida, para no aplicar precios o existencias desactualizados.

Necesidades: BN03. **Reglas:** BR01/BR02/BR03. **Estado de la historia:** Implementada.

BPMN: P01_G01/P01_T03. **Componente:** src/ferremax/domain/model.py; src/ferremax/application/services.py; src/ferremax/api/cotizaciones.py.

Límite: Precios y condiciones los suministra el vendedor en la solicitud; no hay integración con un proveedor de precios. Se consulta S1, fijada en el servicio.

Criterio	Dado	Cuando	Entonces	Estado
AC-US04-01 - Conversión de versión vencida	una versión de cotización cuyo vencimiento ya pasó	el vendedor intenta convertirla en pedido	se rechaza la conversión con indicación de revalidación, sin crear pedido ni reserva	Implementada

Criterio	Dado	Cuando	Entonces	Estado
AC-US04-02 - Nueva versión con historial	una cotización vencida y nuevos precios, condiciones y vigencia válidos	el vendedor solicita revalidarla	se guarda una nueva versión vigente conservando intacta la anterior y sin crear pedido ni reserva	Implementada
AC-US04-03 - Instante exacto del vencimiento	una cotización cuyo vencimiento es el instante actual	se intenta convertir esa versión	se rechaza por falta de vigencia sin alterar los datos	Implementada
AC-US04-04 - Revalidaciones simultáneas	dos solicitudes concurrentes de revalidación para la misma versión vencida	ambas intentan guardar la versión siguiente	solo se conserva una versión nueva y la otra solicitud recibe un rechazo sin duplicar efectos	Implementada

Pruebas y límites de los criterios:

• AC-US04-01: tests/test_api_conversion.py, método test_version_1_vencida_rechaza_antes_de_crear_pedido.

• AC-US04-02: tests/test_api_revalidacion_postgres.py, método test_http_persiste_dos_versiones_auditoria_y_evento_sin_pedido_ni_reserva.

• AC-US04-03: tests/test_api_conversion.py, método test_fuera_de_vigencia_rechaza_sin_cambios.

• AC-US04-04: tests/test_api_revalidacion_postgres.py, método test_dos_revalidaciones_concurrentes_crean_una_sola_version.

US05 - Convertir una cotización aceptada en pedido

Como vendedor, quiero convertir una cotización aceptada en pedido, para evitar recapturar información y mantener trazabilidad.

Necesidades: BN03. **Reglas:** BR01/BR02/BR03. **Estado de la historia:** Implementada.

BPMN: P01_T04/P01_G02/P01_T05. **Componente:** src/ferremax/domain/model.py; src/ferremax/application/services.py; src/ferremax/api/cotizaciones.py.

Límite: La conversión crea Pendiente y consulta disponibilidad de S1; una disponibilidad insuficiente no impide crear el pedido pendiente, pero sí su confirmación.

Criterio	Dado	Cuando	Entonces	Estado
AC-US05-01 - Conversión válida	una versión vigente de cotización que el cliente acepta	el vendedor solicita convertirla	se crea un pedido Pendiente ligado al folio y versión, conserva partidas y total, consulta disponibilidad y no reserva inventario	Implementada

Criterio	Dado	Cuando	Entonces	Estado
AC-US05-02 - Aceptación obligatoria	una versión vigente que el cliente no ha aceptado	el vendedor solicita convertirla	se rechaza la conversión sin crear pedido ni reserva	Implementada
AC-US05-03 - Pendiente sin disponibilidad	una cotización vigente y aceptada con un producto registrado pero sin unidades disponibles	se convierte la cotización	se crea un pedido Pendiente y se informa la disponibilidad en 0 sin crear reserva	Implementada

Pruebas y límites de los criterios:

• AC-US05-01: tests/test_api_conversion_postgres.py, método test_pedido_http_persiste_version_2_total_240_y_evento_sin_reserva.

• AC-US05-02: tests/test_api_conversion.py, método test_sin_aceptacion_rechaza_una_version_vigente.

• AC-US05-03: tests/test_api_conversion_postgres.py, método test_sin_stock_se_crea_pendiente_pero_no_reserva.

US06 - Registrar el pago de una venta de contado

Como Caja, quiero registrar el pago de una venta de contado, para confirmar el pedido solo con un pago autorizado.

Necesidades: BN02/BN04. **Reglas:** BR05. **Estado de la historia:** Simulada.

BPMN: P02_T02/P02_G02. **Componente:** src/ferremax/domain/model.py; src/ferremax/application/services.py; src/ferremax/api/pedidos.py.

Límite: El cuerpo HTTP proporciona pago_autorizado y referencia. No se procesa dinero ni se integra una pasarela; el actor Caja también es de demo.

Criterio	Dado	Cuando	Entonces	Estado
AC-US06-01 - Pago autorizado y stock suficiente	un pedido Pendiente, referencia de pago autorizado y disponibilidad suficiente	Caja confirma el pedido desde una ubicación	se guardan Confirmado, su reserva, auditoría y evento en una sola transacción	Simulada
AC-US06-02 - Pago rechazado	un pedido Pendiente y un pago no autorizado	Caja intenta confirmarlo	el pedido permanece Pendiente sin reserva ni evento de confirmación nuevo	Simulada
AC-US06-03 - Referencia obligatoria	un pedido Pendiente sin referencia de pago válida	Caja intenta confirmarlo	se rechaza la solicitud y se conserva el estado anterior	Simulada

Pruebas y límites de los criterios:

• AC-US06-01: tests/test_api_confirmacion_postgres.py, método test_confirmacion_http_persiste_estado_reserva_auditoria_y_evento.

• AC-US06-02: tests/test_api_confirmacion_postgres.py, método test_pago_no_autorizado_no_reserva_ni_confirma.

• AC-US06-03: tests/test_api_confirmacion.py, método test_datos_invalidos_se_rechazan_sin_cambios.

US07 - Validar automáticamente las condiciones de crédito de un cliente empresarial

Como analista de crédito, quiero validar automáticamente las condiciones de crédito de un cliente empresarial, para reducir riesgos y tiempos de autorización.

Necesidades: BN05. **Reglas:** BR09/BR10. **Estado de la historia:** Parcial.

BPMN: P03_T01/P03_G01/P03_T02/P03_G02. **Componente:** src/ferremax/domain/model.py.

Límite: CuentaCredito y PoliticaCredito evalúan condiciones básicas, pero no hay flujo de solicitud, persistencia, decisión auditada ni endpoint de crédito.

Criterio	Dado	Cuando	Entonces	Estado
AC-US07-01 - Crédito normal suficiente	un cliente empresarial activo, sin bloqueo y con crédito disponible suficiente en la misma moneda	solicita comprar a crédito	se autoriza la condición normal y se registra la evaluación	Diseñada
AC-US07-02 - Cliente bloqueado	un cliente empresarial bloqueado	se evalúa una solicitud de crédito	se obtiene EnRevision y no se confirma automáticamente el pedido	Parcial

Pruebas y límites de los criterios:

• AC-US07-01: sin prueba integral identificada. El cálculo aislado no acredita registro ni autorización integral de un pedido.

• AC-US07-02: tests/test_architecture.py, método ArquitecturaTest::test_credito_bloqueado_insuficiente_inactivo. La prueba acredita EnRevision en la política aislada, no persistencia del expediente ni un cambio de estado del pedido.

US08 - Resolver una excepción de crédito con motivo

Como responsable de Caja/Crédito, quiero resolver una excepción de crédito con motivo, para controlar decisiones fuera de la política normal.

Necesidades: BN05. **Reglas:** BR11. **Estado de la historia:** Diseñada.

BPMN: P03_T04/P03_G03/P03_T05. **Componente:** Sin caso de uso implementado.

Límite: Faltan permisos nominales y límites de autorización comercial; no se inventan umbrales de excepción.

Criterio	Dado	Cuando	Entonces	Estado
AC-US08-01 - Excepción autorizada	una solicitud de crédito en revisión	el responsable autoriza una excepción e indica el motivo	se registran decisión, responsable, fecha y condición autorizada	Diseñada
AC-US08-02 - Excepción rechazada	una solicitud de crédito en revisión	el responsable rechaza la excepción con motivo	el pedido no se confirma y el vendedor recibe el motivo	Diseñada

Pruebas y límites de los criterios:

• AC-US08-01: sin prueba integral identificada.

• AC-US08-02: sin prueba integral identificada.

US09 - Reservar inventario al confirmar un pedido

Como sistema, quiero reservar inventario al confirmar un pedido, para impedir que otra venta use la misma cantidad.

Necesidades: BN02. **Reglas:** BR04/BR05. **Estado de la historia:** Implementada.

BPMN: P02_T03/P02_B01. **Componente:** src/ferremax/domain/model.py; src/ferremax/application/services.py; src/ferremax/infrastructure/postgres.py.

Límite: Un pedido reserva desde una sola ubicación. La reserva, estado, auditoría y evento comparten transacción; un fallo revierte todos los cambios.

Criterio	Dado	Cuando	Entonces	Estado
AC-US09-01 - Reserva atómica	un pedido Pendiente con pago autorizado y unidades suficientes	se confirma el pedido	la reserva propia y el estado Confirmado se guardan juntos y el disponible disminuye por lo reservado	Implementada
AC-US09-02 - Competencia por la última unidad	dos pedidos con autorización de pago compiten por la última unidad disponible	ambos intentan confirmarse concurrentemente	solo uno obtiene la reserva y el otro recibe rechazo por disponibilidad, permaneciendo Pendiente	Implementada
AC-US09-03 - Reserva exactamente igual al disponible	3 unidades disponibles y un pedido de 3 unidades con pago autorizado	se confirma ese pedido	se reservan 3 unidades y el disponible queda en 0	Implementada
AC-US09-04 - Solicitud mayor que el disponible	3 unidades disponibles y un pedido de 4 unidades con pago autorizado	se intenta confirmar el pedido	se rechaza y el pedido almacenado permanece Pendiente sin alterar reservas ni registrar un éxito	Implementada

Criterio	Dado	Cuando	Entonces	Estado
AC-US09-05 - Fallo en la segunda partida	un pedido con dos productos y disponibilidad insuficiente en el segundo	la confirmación intenta reservar ambas partidas	se revierte la reserva del primer producto junto con estado, referencia de pago y efectos de éxito	Implementada
AC-US09-06 - Confirmación repetida	un pedido ya Confirmado con su reserva persistida	se vuelve a solicitar su confirmación	se rechaza sin duplicar reserva ni evento ni reemplazar la referencia original	Implementada

Pruebas y límites de los criterios:

• AC-US09-01: tests/test_api_confirmacion_postgres.py, método test_confirmacion_http_persiste_estado_reserva_auditoria_y_evento.

• AC-US09-02: tests/test_api_confirmacion_postgres.py, método test_dos_pedidos_compiten_por_ultima_unidad_desde_http.

• AC-US09-03: tests/test_e9_inventario.py, método InventarioE9Test::test_e9_inv_03_limite_reservar_tres_agota_el_disponible.

• AC-US09-04: tests/test_e9_inventario.py, método InventarioE9Test::test_e9_inv_02_error_pedir_cuatro_conserva_pedido_e_inventario.

• AC-US09-05: tests/test_api_confirmacion_postgres.py, método test_segundo_sku_sin_stock_revierte_primera_reserva.

• AC-US09-06: tests/test_api_confirmacion_postgres.py, método test_confirmar_dos_veces_no_duplica_reserva_ni_evento.

US10 - Cancelar o reducir un pedido antes del surtido

Como vendedor, quiero cancelar o reducir un pedido antes del surtido, para liberar mercancía que ya no necesita el cliente.

Necesidades: BN02. **Reglas:** BR06. **Estado de la historia:** Parcial.

BPMN: P02_T05/P02_G04/P02_G05/P02_T06/P02_T08. **Componente:** src/ferremax/domain/model.py; src/ferremax/application/services.py; src/ferremax/api/pedidos.py.

Límite: Se implementó cancelación completa; no reducción de partidas. El actor autorizado es simulado y surtido es una guarda, sin operación de surtido en la API.

Criterio	Dado	Cuando	Entonces	Estado
AC-US10-01 - Cancelación libera solo lo propio	un pedido Confirmado, aún no surtido, con autorización y motivo, cuya reserva es de 2 unidades y comparte inventario con 7 unidades reservadas para otro pedido	se cancela el pedido	se liberan solo sus 2 unidades, se conservan las 7 ajenas y se guarda motivo, auditoría y evento	Implementada
AC-US10-02 - Reducción libera diferencia	un pedido Confirmado y aún no surtido con una partida reservada	se autoriza reducir la cantidad de esa partida con motivo	se libera solo la diferencia y se conserva el resto de la reserva y su trazabilidad	Diseñada
AC-US10-03 - Rechazo después del surtido	un pedido Confirmado marcado como surtido	se solicita cancelarlo con motivo	se rechaza conservando estado, reservas y registros previos	Implementada
AC-US10-04 - Cancelación repetida	un pedido ya Cancelado y sin su reserva activa	se solicita cancelarlo de nuevo	se rechaza sin liberar cantidades adicionales ni duplicar el evento de cancelación	Implementada
AC-US10-05 - Liberación fallida revierte todo	un pedido con reserva de dos productos y una inconsistencia que impide liberar el segundo	se intenta cancelar el pedido	se revierte la primera liberación y se conservan estado y registros originales	Implementada

Pruebas y límites de los criterios:

• AC-US10-01: tests/test_api_cancelacion_postgres.py, método test_cancela_y_persiste_liberacion_conservando_otro_pedido.

• AC-US10-02: sin prueba integral identificada. Es una separación del criterio original; no se elimina ni se declara implementada la reducción.

• AC-US10-03: tests/test_e9_cancelacion.py, método CancelacionE9Test::test_e9_can_02_error_pedido_surtido_conserva_estado_reservas_y_eventos.

• AC-US10-04: tests/test_api_cancelacion_postgres.py, método test_rechazos_conservan_datos_y_no_agregan_cancelaciones.

• AC-US10-05: tests/test_api_cancelacion_postgres.py, método test_fallo_al_liberar_segundo_producto_revierte_toda_la_transaccion.

US11 - Solicitar una transferencia desde otra ubicación

Como vendedor, quiero solicitar una transferencia desde otra ubicación, para surtir un pedido sin prometer inventario inexistente.

Necesidades: BN04/BN06. **Reglas:** BR07/BR12. **Estado de la historia:** Diseñada.

BPMN: P04_T01/P04_T02/P04_T03. **Componente:** Sin flujo implementado.

Límite: Consultar otras ubicaciones no implementa selección de origen, transferencia ni reserva multiorigen.

Criterio	Dado	Cuando	Entonces	Estado
AC-US11-01 - Solicitud trazable	una sucursal sin unidades suficientes y otra ubicación con disponible	el vendedor solicita una transferencia tras consultar primero almacén central y después otras sucursales	se registra un folio con origen, destino, partidas, cantidades, pedido relacionado y estado Solicitado	Diseñada

Pruebas y límites de los criterios:

• AC-US11-01: sin prueba integral identificada.

US12 - Autorizar, rechazar y despachar una transferencia

Como responsable de almacén, quiero autorizar, rechazar y despachar una transferencia, para controlar la mercancía que cambia de ubicación.

Necesidades: BN06. **Reglas:** BR12. **Estado de la historia:** Diseñada.

BPMN: P04_T04/P04_G03/P04_T05. **Componente:** Sin flujo implementado.

Límite: No se mueven cantidades en tránsito mediante la API actual.

Criterio	Dado	Cuando	Entonces	Estado
AC-US12-01 - Despacho autorizado	una transferencia autorizada y preparada	almacén registra su salida	la cantidad se registra en tránsito y se conservan responsable y fecha	Diseñada
AC-US12-02 - Rechazo por faltante	una solicitud cuyo origen ya no tiene suficiente disponible	almacén revisa la solicitud	se rechaza con motivo y el pedido relacionado vuelve a evaluación	Diseñada

Pruebas y límites de los criterios:

• AC-US12-01: sin prueba integral identificada.

• AC-US12-02: sin prueba integral identificada.

US13 - Confirmar la recepción de una transferencia

Como almacén destino, quiero confirmar la recepción de una transferencia, para actualizar inventario utilizable y cerrar el movimiento.

Necesidades: BN06. **Reglas:** BR13. **Estado de la historia:** Diseñada.

BPMN: P04_T06/P04_G04/P04_T07/P04_T08. **Componente:** Sin flujo implementado.

Límite: Los valores de tránsito del demo son datos iniciales; no acreditan recepción de una transferencia.

Criterio	Dado	Cuando	Entonces	Estado
AC-US13-01 - Recepción completa	una transferencia en tránsito con todas sus cantidades recibidas	destino confirma la recepción	disminuye tránsito, aumenta existencia física utilizable en destino y queda Recibida sin ocultar reservas ajenas	Diseñada
AC-US13-02 - Diferencia de recepción	una transferencia cuya cantidad recibida difiere de la esperada	destino registra lo efectivamente recibido	queda una incidencia trazable y no se oculta la diferencia	Diseñada

Pruebas y límites de los criterios:

• AC-US13-01: sin prueba integral identificada.

• AC-US13-02: sin prueba integral identificada.

US14 - Proponer y registrar un surtido parcial

Como vendedor, quiero proponer y registrar un surtido parcial, para permitir que el cliente decida sobre faltantes.

Necesidades: BN04/BN07. **Reglas:** BR08. **Estado de la historia:** Diseñada.

BPMN: P04_T09/P04_G05/P04_T10/P04_T11. **Componente:** Sin flujo implementado.

Límite: Aceptar una cotización no equivale a aceptar un parcial; faltan cantidades y estados por partida.

Criterio	Dado	Cuando	Entonces	Estado
AC-US14-01 - Aceptación de parcial	un pedido con disponibilidad insuficiente	el cliente acepta expresamente las cantidades surtibles	se separan las cantidades surtibles y pendientes por partida y se conserva la aceptación	Diseñada
AC-US14-02 - Rechazo de parcial	un pedido incompleto cuyo cliente no acepta el parcial	se registra su decisión de esperar o cancelar	se conserva la decisión y se libera únicamente lo que corresponda si hay una cancelación autorizada	Diseñada

Pruebas y límites de los criterios:

• AC-US14-01: sin prueba integral identificada.

• AC-US14-02: sin prueba integral identificada.

US15 - Generar y completar una orden de surtido

Como responsable de almacén, quiero generar y completar una orden de surtido, para preparar exactamente lo reservado para el pedido.

Necesidades: BN04/BN07. **Reglas:** BR05/BR07/BR08. **Estado de la historia:** Diseñada.

BPMN: P05_T01/P05_G01. **Componente:** Solo existe Pedido.surtido como guarda.

Límite: No existe operación para completar surtido. Preparar surtido en una prueba solo verifica la guarda de cancelación.

Criterio	Dado	Cuando	Entonces	Estado
AC-US15-01 - Surtido correcto	un pedido Confirmado con reservas	almacén prepara y verifica sus partidas	se registran cantidades surtidas, faltantes, responsable y hora sin exceder lo reservado	Diseñada

Pruebas y límites de los criterios:

• AC-US15-01: sin prueba integral identificada.

US16 - Programar unidad, operador y ventana de entrega

Como coordinador de entrega, quiero programar unidad, operador y ventana de entrega, para organizar pedidos grandes enviados a domicilio.

Necesidades: BN08. **Reglas:** BR14. **Estado de la historia:** Diseñada.

BPMN: P05_T03/P05_G03/P05_T04. **Componente:** Sin flujo implementado.

Límite: No se implementan transporte ni optimización de rutas; la segunda permanece fuera del alcance original.

Criterio	Dado	Cuando	Entonces	Estado
AC-US16-01 - Programación completa	un pedido surtido con entrega a domicilio y dirección validada	se programa la entrega	se asignan unidad, operador, ventana y estado Programado	Diseñada
AC-US16-02 - Datos incompletos	una entrega sin dirección o contacto válido	se intenta programar la salida	se bloquea y se indican los datos faltantes	Diseñada

Pruebas y límites de los criterios:

• AC-US16-01: sin prueba integral identificada.

• AC-US16-02: sin prueba integral identificada.

US17 - Registrar la entrega o una incidencia

Como operador de entrega, quiero registrar la entrega o una incidencia, para cerrar el pedido con evidencia.

Necesidades: BN08. **Reglas:** BR14. **Estado de la historia:** Diseñada.

BPMN: P05_T05/P05_G04/P05_T06/P05_T07. **Componente:** Sin flujo implementado.

Límite: Los estados del negocio Entregado/Incidencia de esta vista aún no forman parte de EstadoPedido de la aplicación.

Criterio	Dado	Cuando	Entonces	Estado
AC-US17-01 - Entrega completada	una entrega en ruta realizada al destinatario	el operador registra receptor, fecha, hora y evidencia	la entrega y el pedido se cierran como entregados con su trazabilidad	Diseñada
AC-US17-02 - Incidencia en ruta	una entrega que no pudo realizarse	el operador registra motivo y evidencia	la entrega queda con incidencia y el pedido no se cierra como entregado	Diseñada

Pruebas y límites de los criterios:

• AC-US17-01: sin prueba integral identificada.

• AC-US17-02: sin prueba integral identificada.

US18 - Registrar una solicitud de devolución

Como vendedor, quiero registrar una solicitud de devolución, para aplicar la política correcta y mantener trazabilidad.

Necesidades: BN09. **Reglas:** BR15. **Estado de la historia:** Diseñada.

BPMN: P06_T01/P06_G01/P06_T02/P06_G02/P06_T03/P06_T04. **Componente:** Sin flujo implementado.

Límite: La política por categoría y condición sigue pendiente de definición comercial; no se inventa un plazo ni un resultado financiero fijo.

Criterio	Dado	Cuando	Entonces	Estado
AC-US18-01 - Devolución elegible	un comprobante válido y un producto cuya categoría y condición cumplen la política configurada	se autoriza la devolución	se registran la decisión, el resultado financiero y el destino del inventario	Diseñada

Pruebas y límites de los criterios:

• AC-US18-01: sin prueba integral identificada.

US19 - Consultar alertas y reportes operativos

Como gerente de operaciones, quiero consultar alertas y reportes operativos, para detectar riesgos antes de afectar al cliente.

Necesidades: BN10. **Reglas:** BR16. **Estado de la historia:** Diseñada.

BPMN: P07_T04. **Componente:** Sin panel implementado.

Límite: GET de un pedido es una consulta operativa, no el panel agregado de esta historia.

Criterio	Dado	Cuando	Entonces	Estado
AC-US19-01 - Panel operativo	operaciones registradas en las cuatro ubicaciones del negocio	el gerente consulta el panel con filtros de fecha y ubicación	ve cotizaciones por vencer, bajo mínimo, transferencias, crédito en revisión, parciales y entregas pendientes	Diseñada

Pruebas y límites de los criterios:

• AC-US19-01: sin prueba integral identificada.

US20 - Consultar la bitácora de una operación

Como auditor, quiero consultar la bitácora de una operación, para saber quién cambió qué, cuándo y por qué.

Necesidades: BN10. **Reglas:** BR16. **Estado de la historia:** Parcial.

BPMN: P07_T01/P07_T02/P07_G01/P07_T03. **Componente:** src/ferremax/application/services.py; src/ferremax/infrastructure/postgres.py.

Límite: Se almacena auditoría transaccional, pero no hay endpoint de consulta de bitácora, identidad autenticada ni prueba de prohibición de edición por un rol operativo de BD.

Criterio	Dado	Cuando	Entonces	Estado
AC-US20-01 - Consulta de bitácora protegida	una operación sensible modificada	un auditor autorizado consulta su folio	se muestran actor, fecha, hora, acción, valores anterior y nuevo y motivo, sin permitir que un usuario operativo altere la bitácora	Diseñada
AC-US20-02 - Registro transaccional de cambio	un pedido Pendiente, pago autorizado y disponibilidad suficiente	se confirma el pedido	pedido, reserva, auditoría con actor y valores anterior/nuevo y evento se conservan en la misma transacción	Implementada

Pruebas y límites de los criterios:

• AC-US20-01: sin prueba integral identificada. Registrar filas no acredita todavía la consulta protegida completa.

• AC-US20-02: tests/test_api_confirmacion_postgres.py, método test_confirmacion_http_persiste_estado_reserva_auditoria_y_evento. Actor de demostración; este criterio acredita el registro, no autenticación ni inalterabilidad por permisos.

Fuente y evidencia

• Historias y criterios originales de U2.pdf>), conservados sin cambios.

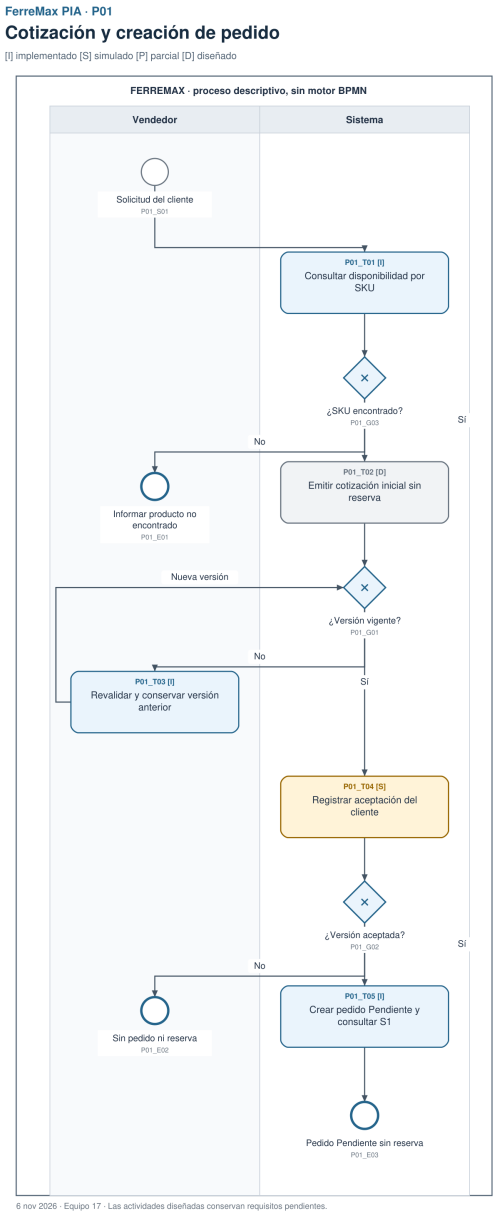
• Evidencia CI/CD del núcleo: registro de 947aeae, 242 pruebas, 0 omisiones y 97.24% de cobertura. Esta métrica no acredita los criterios diseñados ni un porcentaje de historias terminadas.

• Las pruebas referenciadas verifican comportamientos concretos; leer los límites de cada criterio antes de extrapolar sus resultados.

Anexo C. BPMN del proceso de negocio

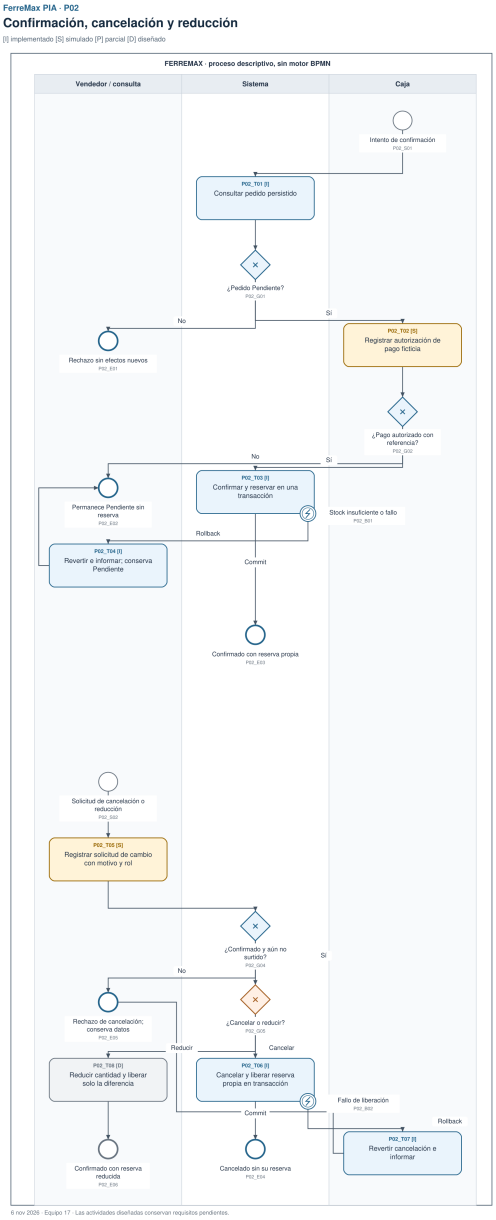
Las siete vistas constituyen un BPMN descriptivo del proceso general. El fuente editable está en docs/analysis/bpmn/ferremax_pia.bpmn. Las actividades distinguen implementación, simulación y diseño pendiente; la existencia de un diagrama no demuestra que todo el proceso se ejecute.

01 cotizacion pedido



Vista BPMN de análisis; consultar leyenda y correspondencia en docs/analysis/BPMN.md.

02 confirmacion cancelacion



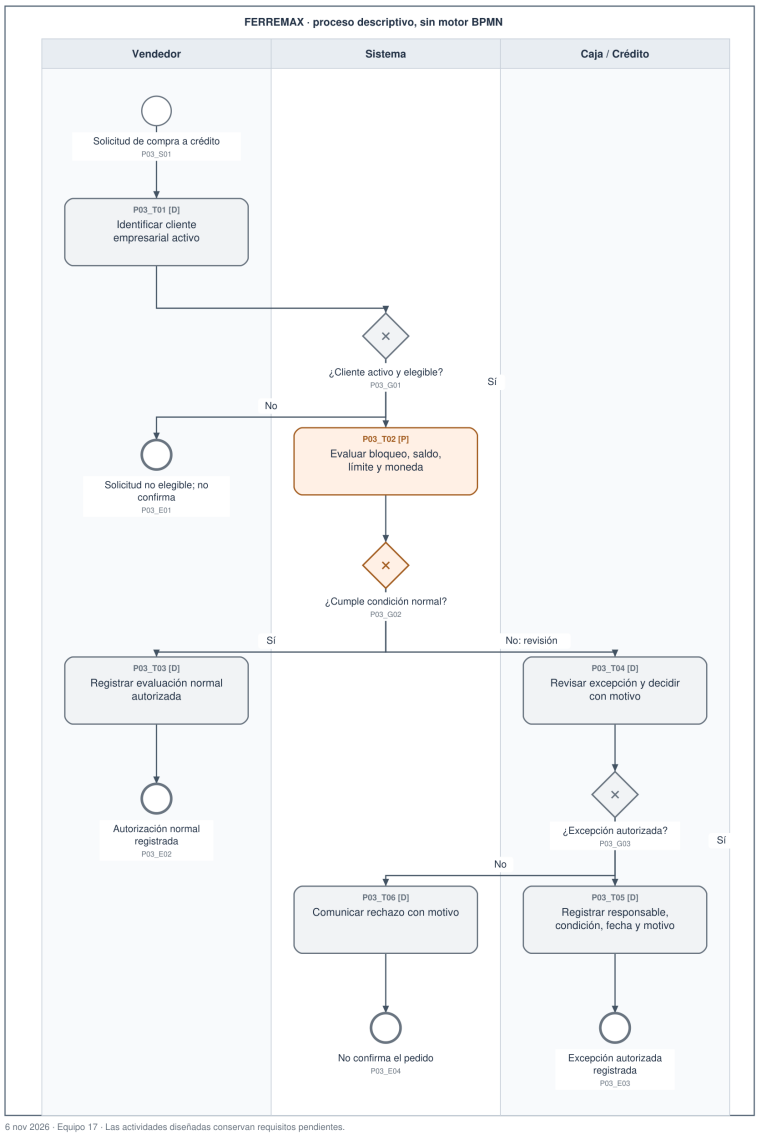
Vista BPMN de análisis; consultar leyenda y correspondencia en docs/analysis/BPMN.md.

03 credito

FerreMax PIA · P03

Crédito normal y excepcional

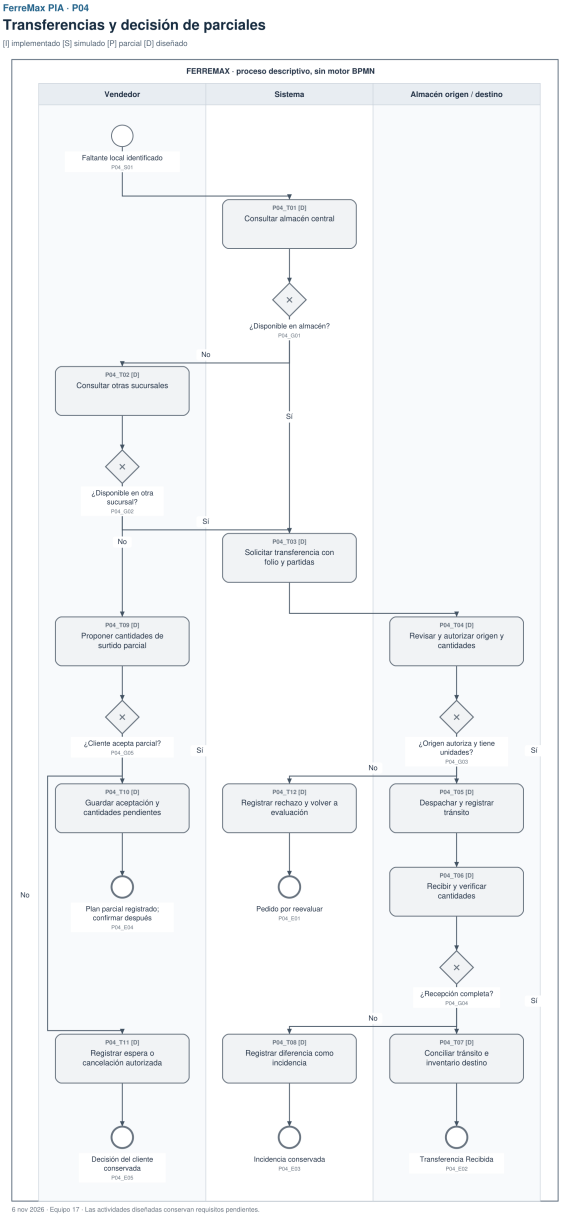
[I] implementado [S] simulado [P] parcial [D] diseñado



6 nov 2026 · Equipo 17 · Las actividades diseñadas conservan requisitos pendientes.

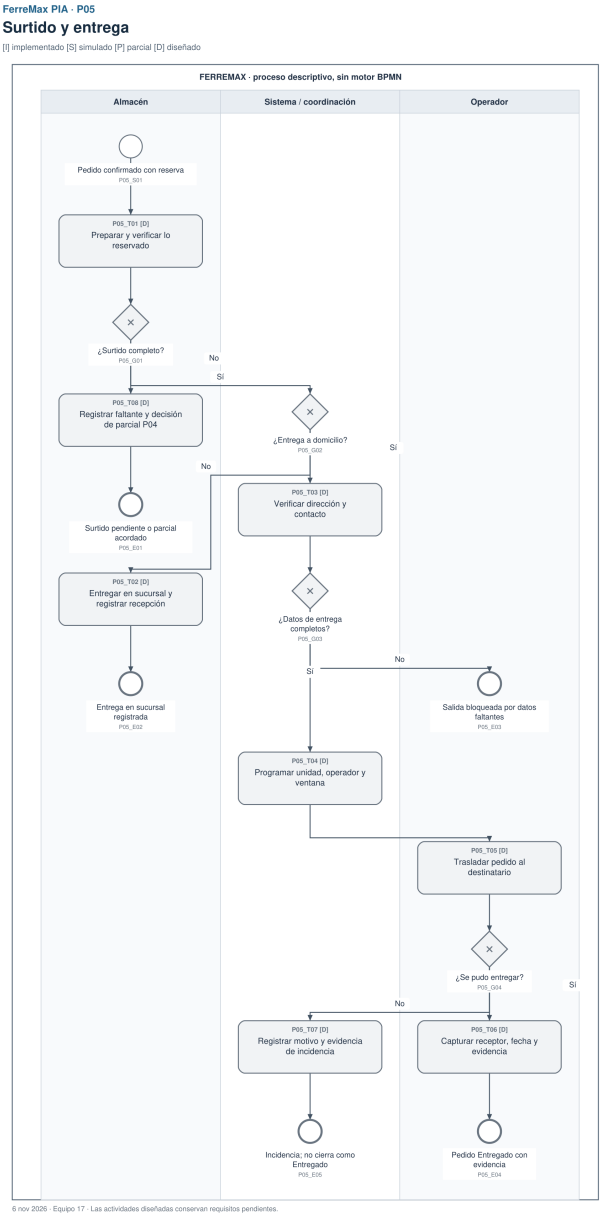
Vista BPMN de análisis; consultar leyenda y correspondencia en docs/analysis/BPMN.md.

04 transferencias parciales



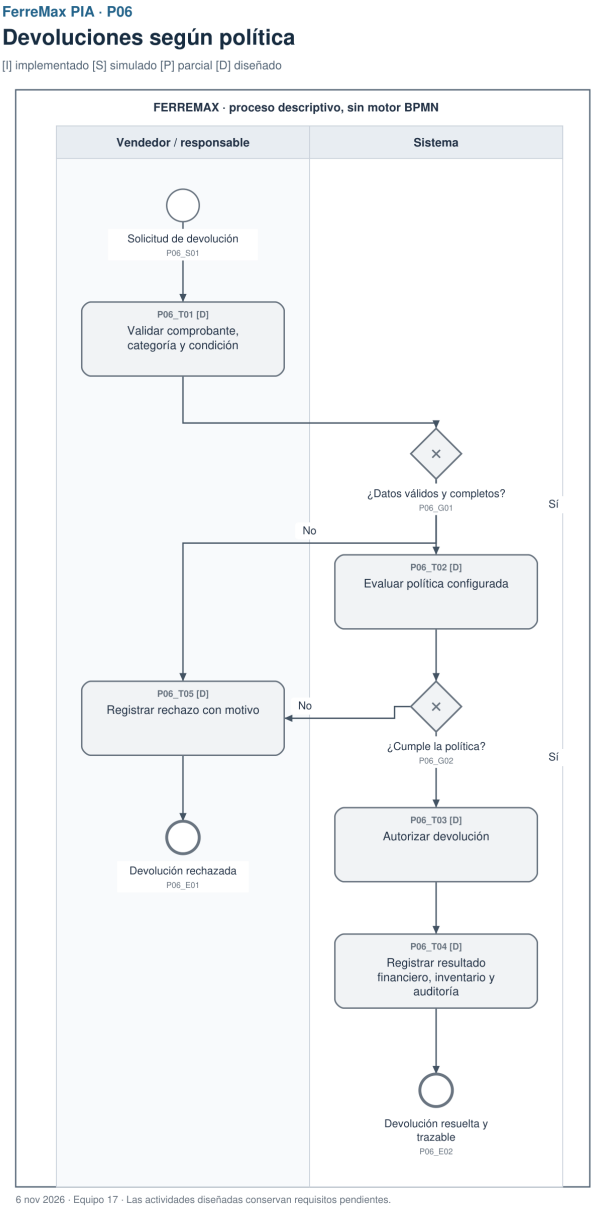
Vista BPMN de análisis; consultar leyenda y correspondencia en docs/analysis/BPMN.md.

05 surtido entrega



Vista BPMN de análisis; consultar leyenda y correspondencia en docs/analysis/BPMN.md.

06 devoluciones



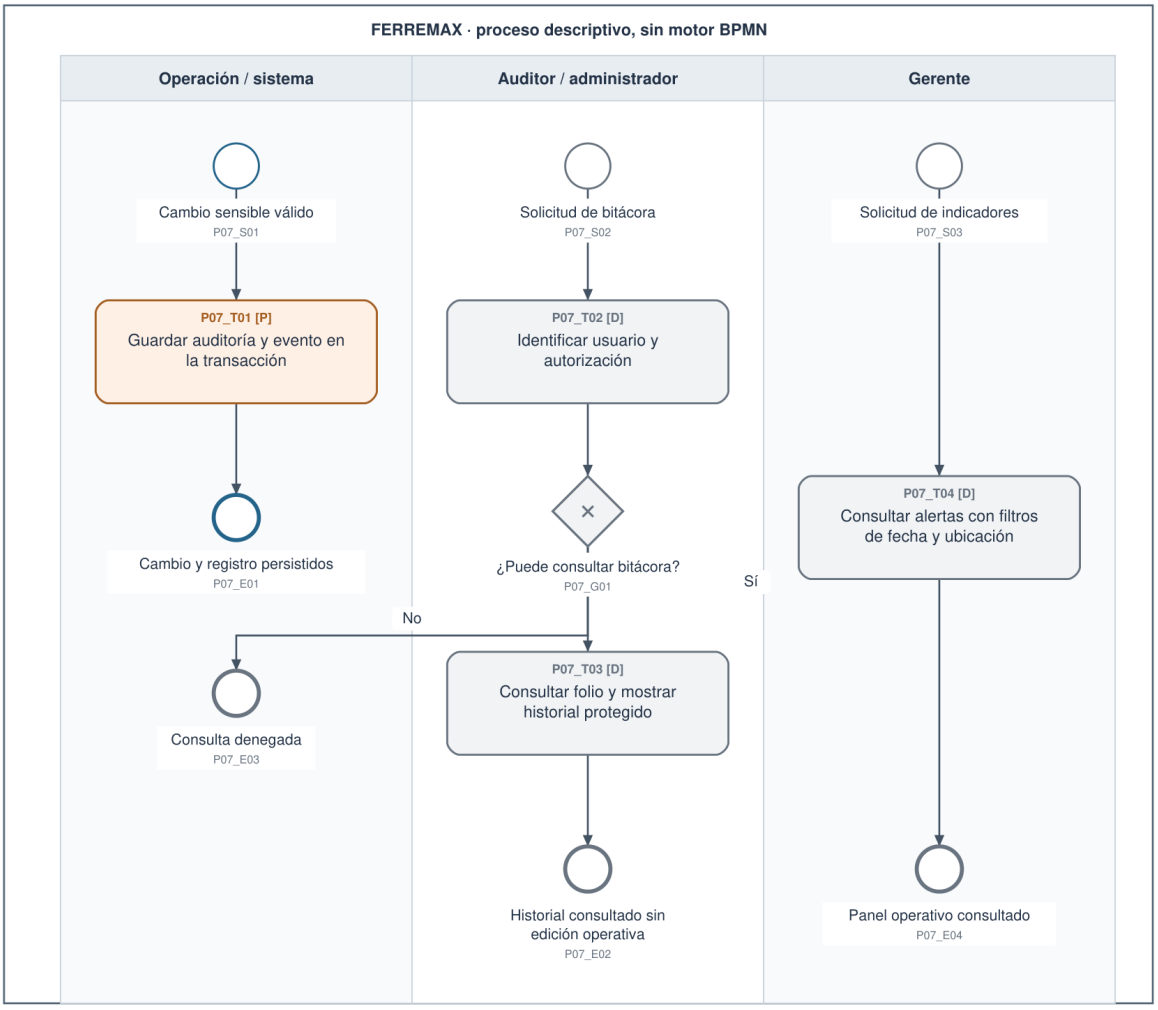
Vista BPMN de análisis; consultar leyenda y correspondencia en docs/analysis/BPMN.md.

07 auditoria reportes

FerreMax PIA · P07

Auditoría y reportes operativos

[I] implementado [S] simulado [P] parcial [D] diseñado



6 nov 2026 · Equipo 17 · Las actividades diseñadas conservan requisitos pendientes.

Vista BPMN de análisis; consultar leyenda y correspondencia en docs/analysis/BPMN.md.

Anexo D. Glosario de dominio

Integración del 6 de noviembre de 2026. Se conserva el vocabulario de U2 y se precisa su relación con el código. Un término del negocio puede estar diseñado sin tener una clase o endpoint implementado.

Términos conservados de U2

Término	Significado compartido	Relación y estado actual
SKU	Identificador de un producto de FerreMax	SKU; consulta HTTP por código. El inventario consultable no es un catálogo maestro independiente.
Producto	Herramienta o material que se cotiza, vende, transfiere o devuelve	Las partidas usan SKU; no hay mantenimiento de un catálogo completo.
Categoría	Agrupación comercial que puede determinar una política	Diseñada, por ejemplo para devoluciones.
Ubicación	Una de las tres sucursales o el almacén central donde se controla inventario	ubicacion_id. Los registros de un SKU pueden existir en algunas ubicaciones; no se generan existencias ficticias en las demás.
Existencia física	Cantidad contabilizada físicamente en una ubicación	InventarioUbicacion.existencia_fisica.
Cantidad comprometida	Suma de reservas activas de pedidos confirmados sobre un inventario	InventarioUbicacion.comprometida; la base guarda también una columna consistente con sus reservas.
Inventario disponible	Cantidad vendible: física menos comprometida, sobre movimientos confirmados	InventarioUbicacion.disponible. No suma tránsito ni permite sobrecomprometer.
Inventario en tránsito	Cantidad despachada por transferencia que todavía no se recibe	en_transito se informa; el flujo de despacho/recepción sigue pendiente.
Stock mínimo	Nivel configurado que genera alerta de reabastecimiento	Diseñado; no existe el panel de alertas.
Cotización	Propuesta con cliente, folio, versión, vigencia, partidas y condiciones	Cotizacion. No crea reserva. Emisión inicial operativa pendiente; revalidación/conversión implementadas.
Vigencia	Intervalo durante el cual puede aceptarse una versión sin revalidarla	Vigencia: emisión <= instante < vencimiento. En el vencimiento exacto ya no está vigente. Fechas con zona horaria.
Revalidación	Nueva revisión de precio, disponibilidad y condiciones de una versión vencida	CotizacionService.revalidar: crea otra versión sin cambiar la anterior. El vendedor suministra precios/condiciones y se consulta S1.
Pedido	Compromiso de compra; en el núcleo deriva de una versión de cotización aceptada	Pedido, con identidad, partidas, origen y estado. La venta directa del análisis no tiene un caso de uso independiente.

Término	Significado compartido	Relación y estado actual
Partida	Renglón numerado que identifica producto, cantidad y precio cuando corresponde	Partida: cantidad positiva y finita; números distintos y moneda homogénea en un documento.
Reserva de inventario	Cantidad comprometida para un pedido confirmado	InventarioUbicacion.reservas, por pedido. Se guarda junto con la confirmación; no por crear una cotización ni un Pendiente.
Surtido	Preparación y verificación física de partidas reservadas	Diseñado; Pedido.surtido solo funciona como guarda de cancelación en el núcleo.
Surtido parcial	Surtido de menos cantidad de la solicitada con aceptación explícita	Diseñado; requiere cantidades surtibles/pendientes por partida y consentimiento diferente de la aceptación de cotización.
Transferencia	Movimiento trazable entre dos ubicaciones	Diseñada; no se acredita por mostrar el campo de tránsito.
Cliente empresarial	Organización registrada que puede tener cuenta de crédito	Diseñado como actor del negocio; no existe registro empresarial operativo completo.
Cuenta de crédito	Límite, saldo, bloqueo y condiciones de un cliente empresarial	CuentaCredito evalúa condiciones básicas; no tiene repositorio ni API integral.
Crédito disponible	Importe que puede utilizarse sin exceder límite y sujeto a bloqueo/condiciones	La evaluación usa límite menos saldo y moneda compatible. Crédito no equivale a inventario disponible.
Excepción de crédito	Decisión motivada de Caja/Crédito fuera de autorización normal	Diseñada; no existe un monto máximo acordado en el caso.
Entrega a domicilio	Traslado de un pedido surtido a dirección validada mediante unidad y operador	Diseñada; no se implementa optimización de rutas.
Evidencia de entrega	Receptor, fecha/hora y soporte del resultado de una entrega	Diseñada; una incidencia no equivale a entrega completada.
Devolución	Solicitud de retorno evaluada contra comprobante, categoría, condición y política	Diseñada; faltan valores de la política y flujo ejecutable.
Bitácora de auditoría	Historial de cambios con actor, fecha/hora, acción, antes/después y motivo	Registro transaccional implementado; consulta protegida e inalterabilidad por permisos operativos pendientes.
Lenguaje ubicuo	Significado consistente entre BRD, historias, BPMN, dominio, API y pruebas	Este glosario y los identificadores compartidos evitan usar términos distintos para la misma regla.

Precisiones incorporadas al integrar el PIA

Término	Significado	Código o evidencia
Versión de cotización	Revisión identificada por el mismo ID/folio y un número de versión	Cotizacion.version; tabla cotizacion_version. El pedido conserva su versión de origen.
Pendiente	Pedido creado que aún no pudo confirmarse	EstadoPedido.PENDIENTE; puede existir con disponible en 0, pero no reserva.
Confirmado	Pedido cuyo cambio de estado y reserva se confirmaron tras validar autorización y existencias	EstadoPedido.CONFIRMADO; el Unit of Work hace persistentes los efectos juntos.
Cancelado	Pedido confirmado cuya cancelación válida liberó su reserva propia	EstadoPedido.CANCELADO; terminal para las operaciones actuales.
Autorización de pago simulada	Referencia y booleano que representan una decisión externa de pago para el ejercicio	AutorizacionPago; no procesa dinero, ni acredita pasarela real.
Reserva propia	Reserva vinculada al ID del pedido que se opera	Cancelar libera esa reserva, sin liberar las de otros pedidos.
Transacción / Unit of Work	Unidad que confirma o revierte conjuntamente los cambios de negocio	PostgresUnitOfWork: pedido, inventario, reservas, auditoría y outbox en una transacción.
Rollback	Reversión de una operación que no pudo confirmarse	Si falla stock, el pedido persistido sigue Pendiente; no es una transición comercial nueva Confirmado → Pendiente.
Evento de dominio	Hecho del negocio registrado después de una operación válida	CotizacionRevalidada, PedidoCreado, PedidoConfirmado, PedidoCancelado.
Outbox	Registro técnico de eventos guardado junto al cambio de negocio	Tabla outbox; sin publicador/consumidor externo implementado. No equivale a una notificación enviada.
Specification	Condición de negocio evaluable y combinable sin efectos secundarios	Vigencia AND aceptación en conversión. OR/NOT son operadores probados, no reglas comerciales adicionales.
Actor de demo	Identificador y rol ficticios que el endpoint proporciona al servicio	Actor.exigir prueba guardas internas; no es una sesión autenticada.
Criterio de aceptación	Condición Given/When/Then que define comportamiento esperado	Archivo aceptacion.feature; es contrato del análisis. No acredita ejecución BDD sin un runner y resultados reales.

Correspondencias y distinciones de uso

• Una **cotización** conserva condiciones; un **pedido Pendiente** conserva la intención aceptada; una **reserva** aparece al confirmar transaccionalmente.

• **Disponible**, **físico**, **comprometido** y **tránsito** son cantidades distintas. Con físico 10 y comprometido 7, disponible es 3 aunque exista mercancía en tránsito.

• **Aceptar cotización**, **autorizar pago**, **autorizar crédito** y **aceptar un parcial** son decisiones distintas; ninguna sustituye automáticamente a otra.

• **Cancelación** libera todo lo reservado por ese pedido; **reducción** libera una diferencia. Solo la primera está implementada.

• **Evento**, **auditoría** y **log técnico** tienen propósitos distintos; registrar outbox no prueba envío externo ni una interfaz de auditoría.

Fuentes: glosario U2.pdf>), BRD integrado, historias, src/ferremax/domain/model.py, servicios y pruebas del núcleo. El original U2 permanece intacto.